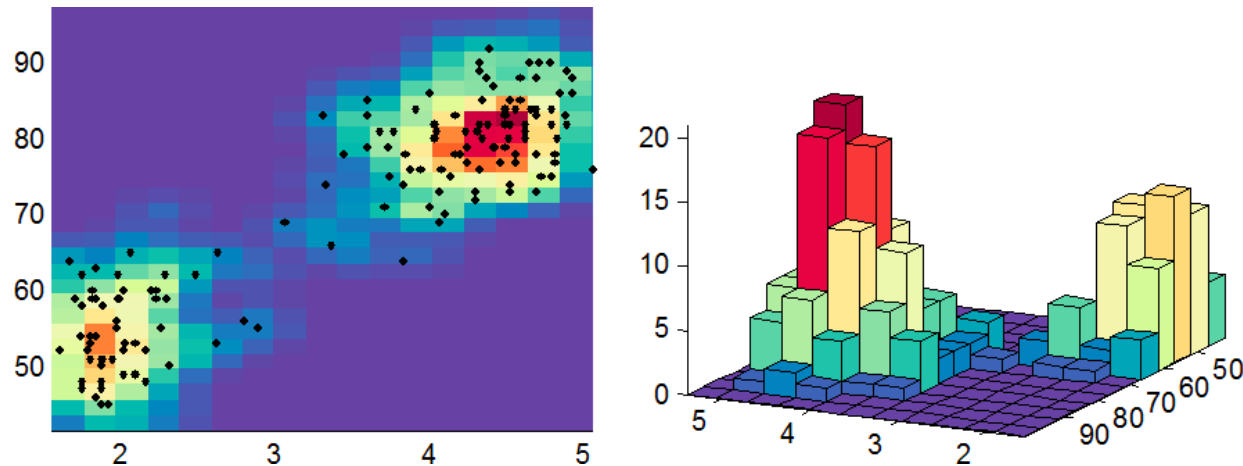


# DIMENSIONALITY REDUCTION AND VISUALIZATION

---

# Histogram estimation in N- dimension

- Cut the space into N-dimensional cube
  - How many cubes are there?
  - Assume I want around 10 samples per cube to be able to estimate a nice distribution without overfitting. How many more samples do I need per one additional dimension?



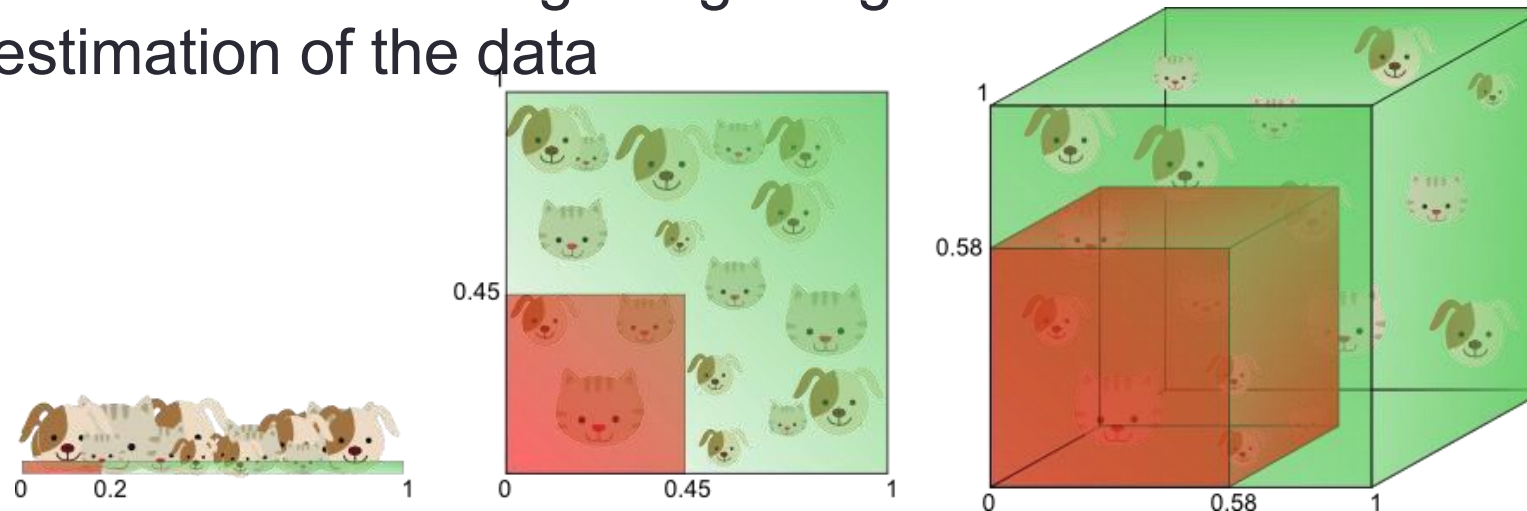
# The Curse of Dimensionality

- Harder to visualize or see structure of
  - Verifying that data come from a straight line/plane needs  $n+1$  data points
- Hard to search in high dimension – More runtime
- Need more data to get a good estimation of the data

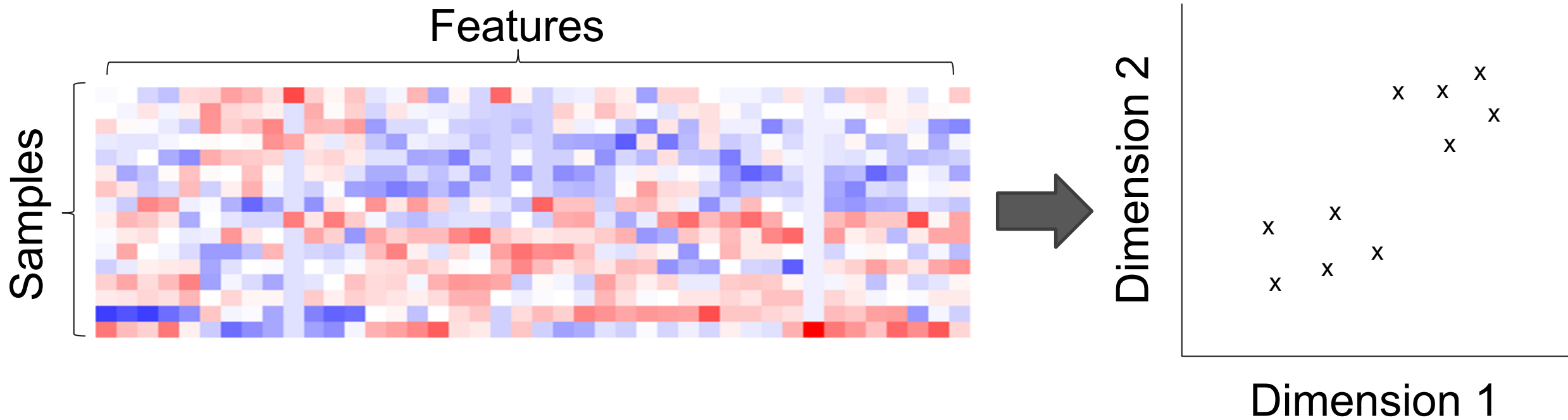
Wind in the morning  $X \in \{\text{Calm, Windy}\}$   
PM2.5 level in the afternoon  $Y \in \{\text{Low, Med, High}\}$   
PM2.5 level in the evening  $Z \in \{\text{Low, Med, High}\}$   
 $\text{argmax } P(Z | Y, X) = \text{argmax } P(Y, X | Z) P(Z)$

| Day | X | Y | Z |
|-----|---|---|---|
| 1   | W | L | M |
| 2   | C | M | M |
| 3   | W | H | M |
| 4   | W | M | H |
| 5   | C | M | L |
| 6   | W | M | L |
| 7   | C | L | H |
| 8   | W | H | L |

| count(Z,Y,X) | Z=L | Z=M | Z=H |
|--------------|-----|-----|-----|
| X=W,Y=L      | 0   | 1   | 0   |
| X=W,Y=M      | 1   | 0   | 1   |
| X=W,Y=H      | 1   | 1   | 0   |
| X=C,Y=L      | 0   | 0   | 1   |
| X=C,Y=M      | 1   | 1   | 0   |
| X=C,Y=H      | 0   | 0   | 0   |

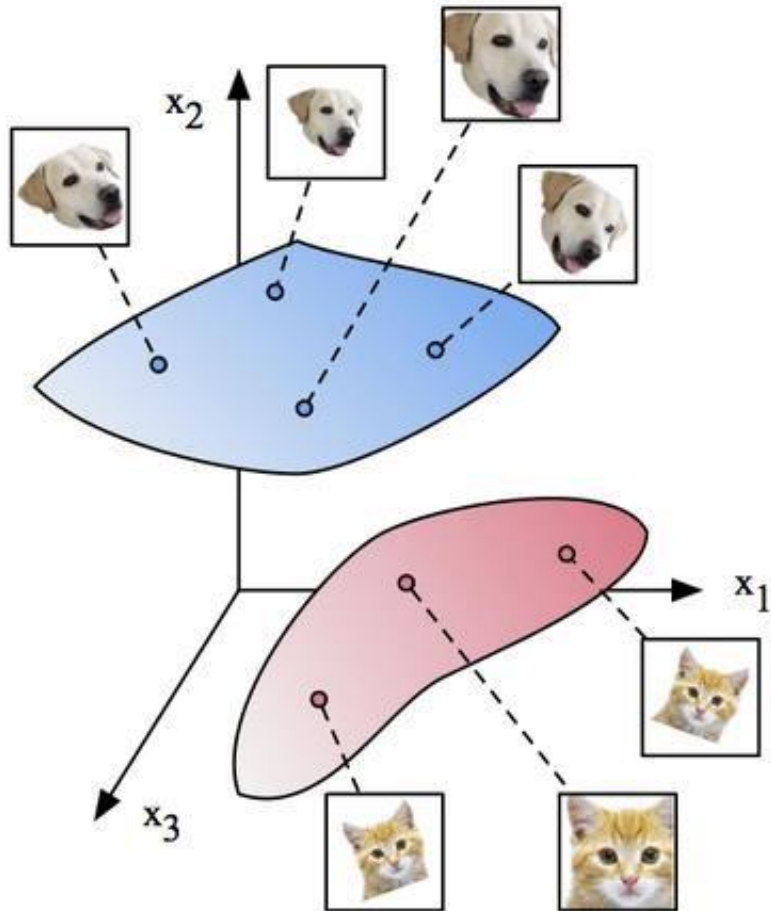


# Dimensionality reduction

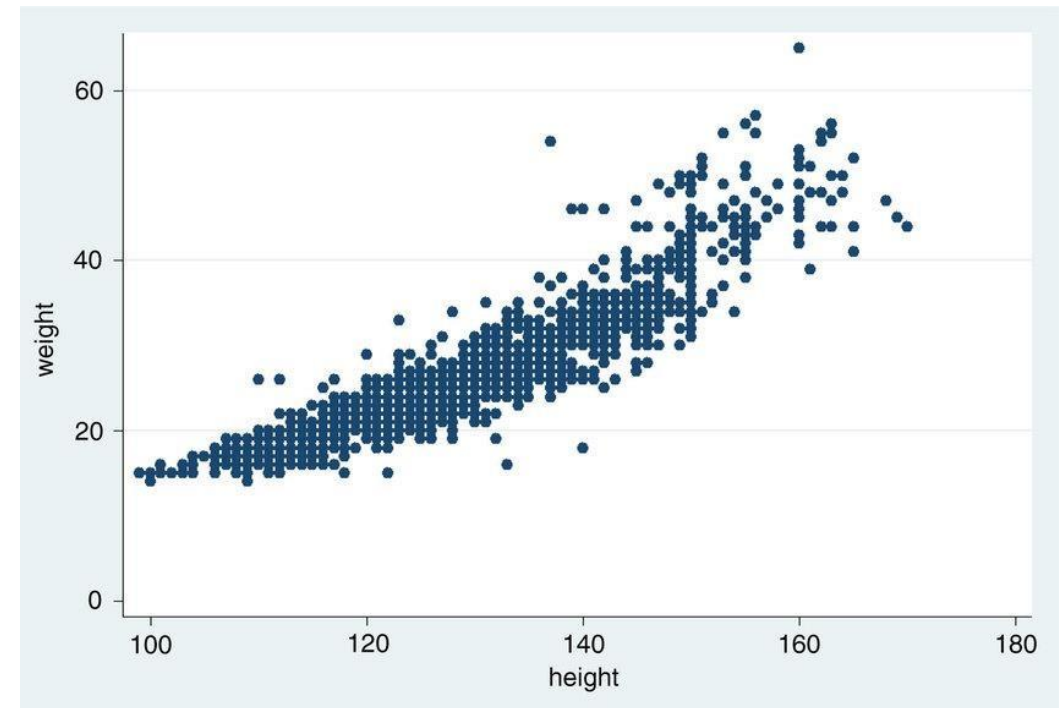


- Summarizing high-dimensional data for visualization
- Based on variance or sample-sample similarity

# Manifold hypothesis



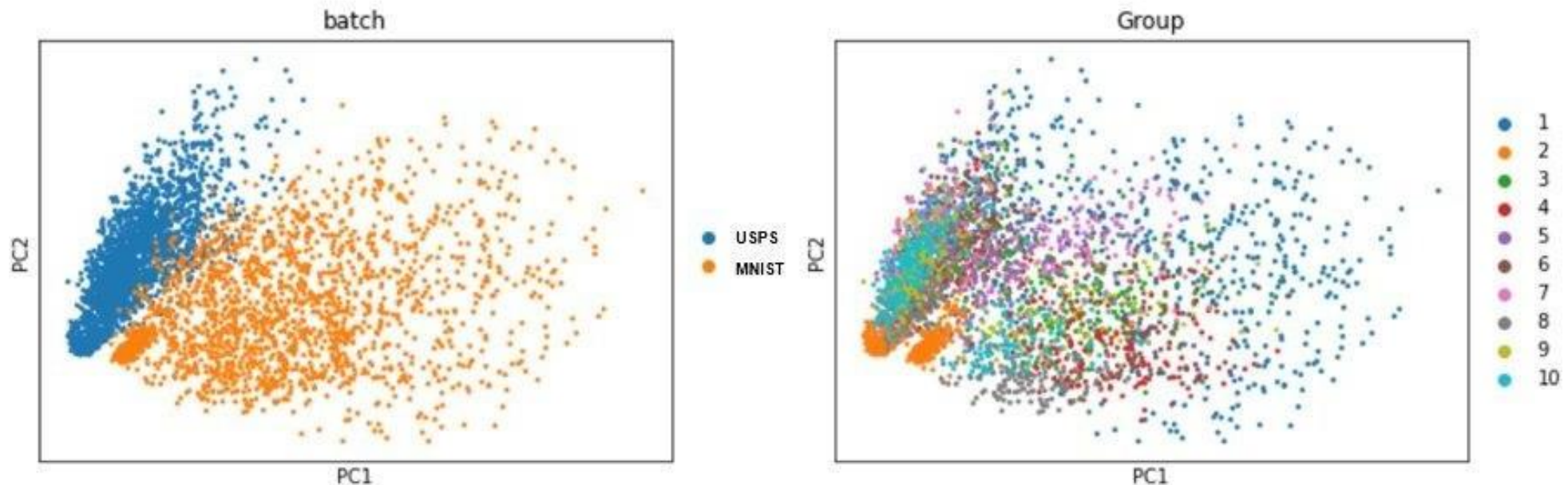
“Real-world, high-dimensional data lie on some low-dimensional manifolds”



# An example: Digit datasets

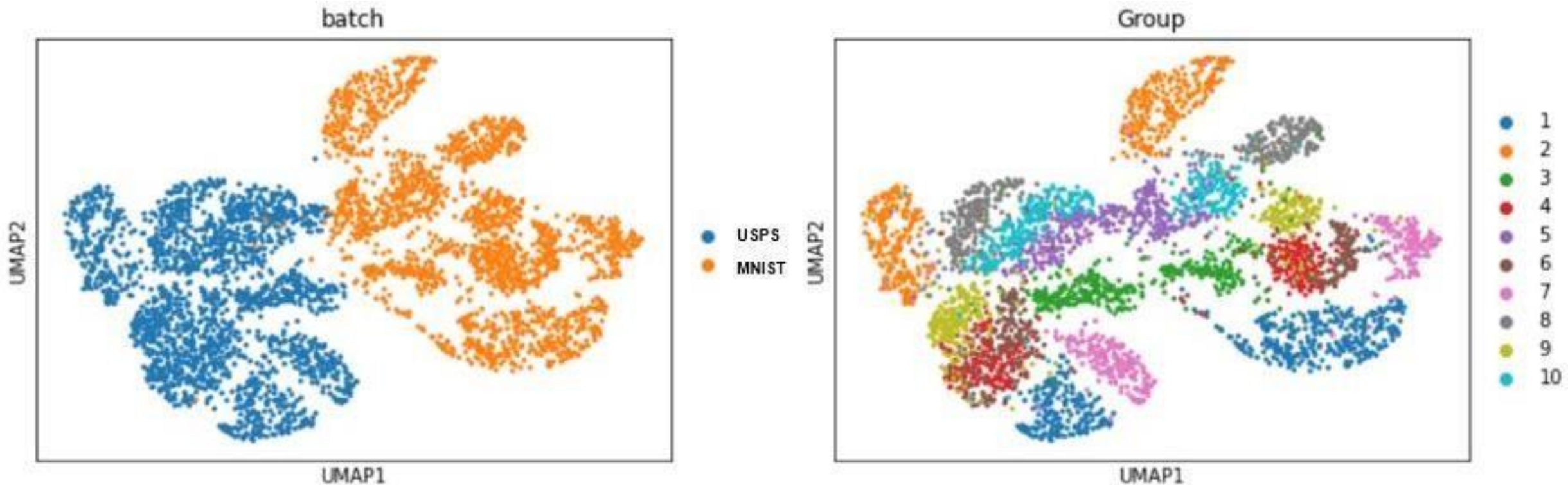


\*adapted from doi:10.1109/TKDE.2017.2669193





# A powerful 2D visualization

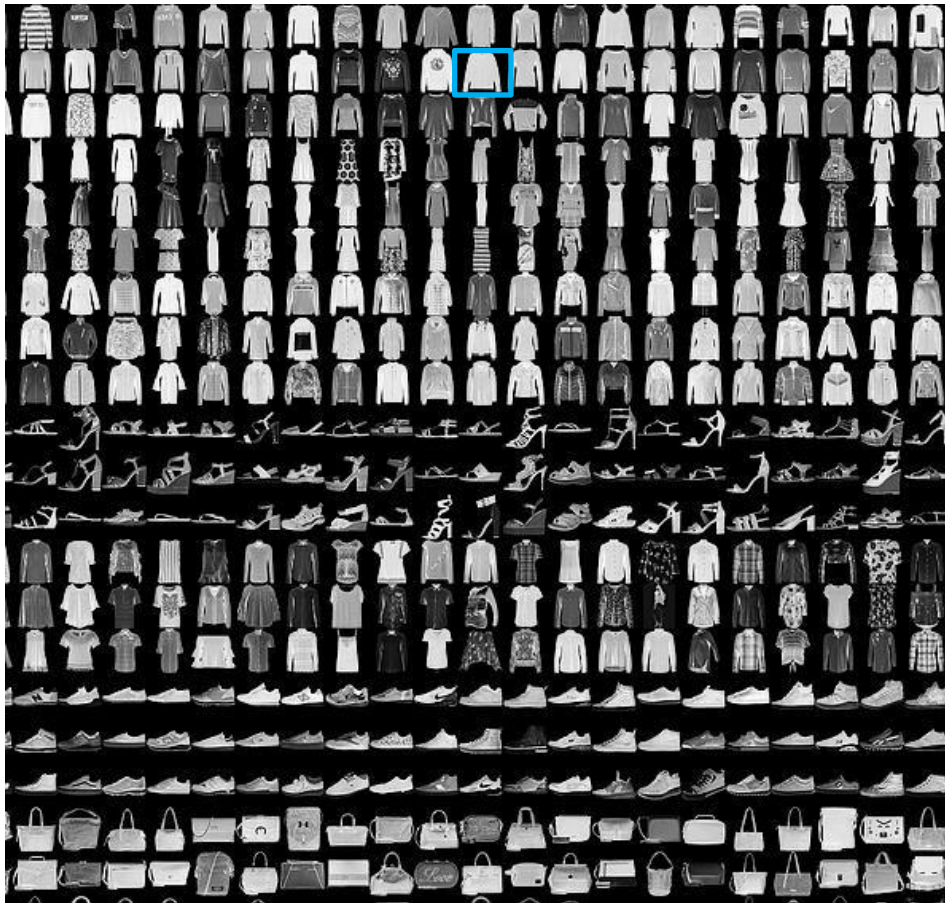


<https://twitter.com/lkmklsmn/status/1436357177887895555>

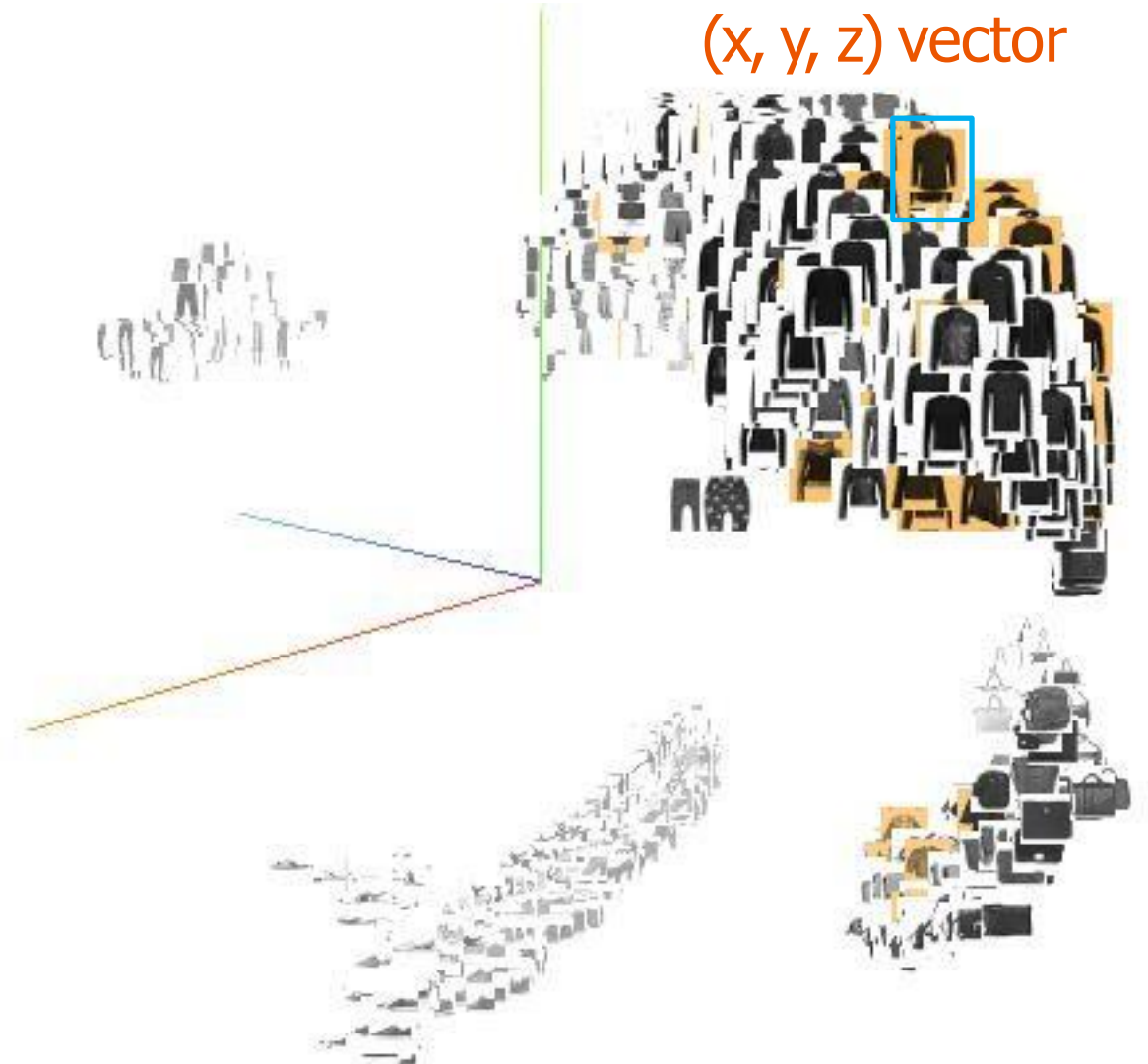
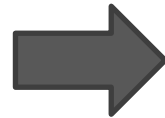
- Both data source and digit identity can be distinguished

# Another example: Fashion MNIST

28x28 pixel image



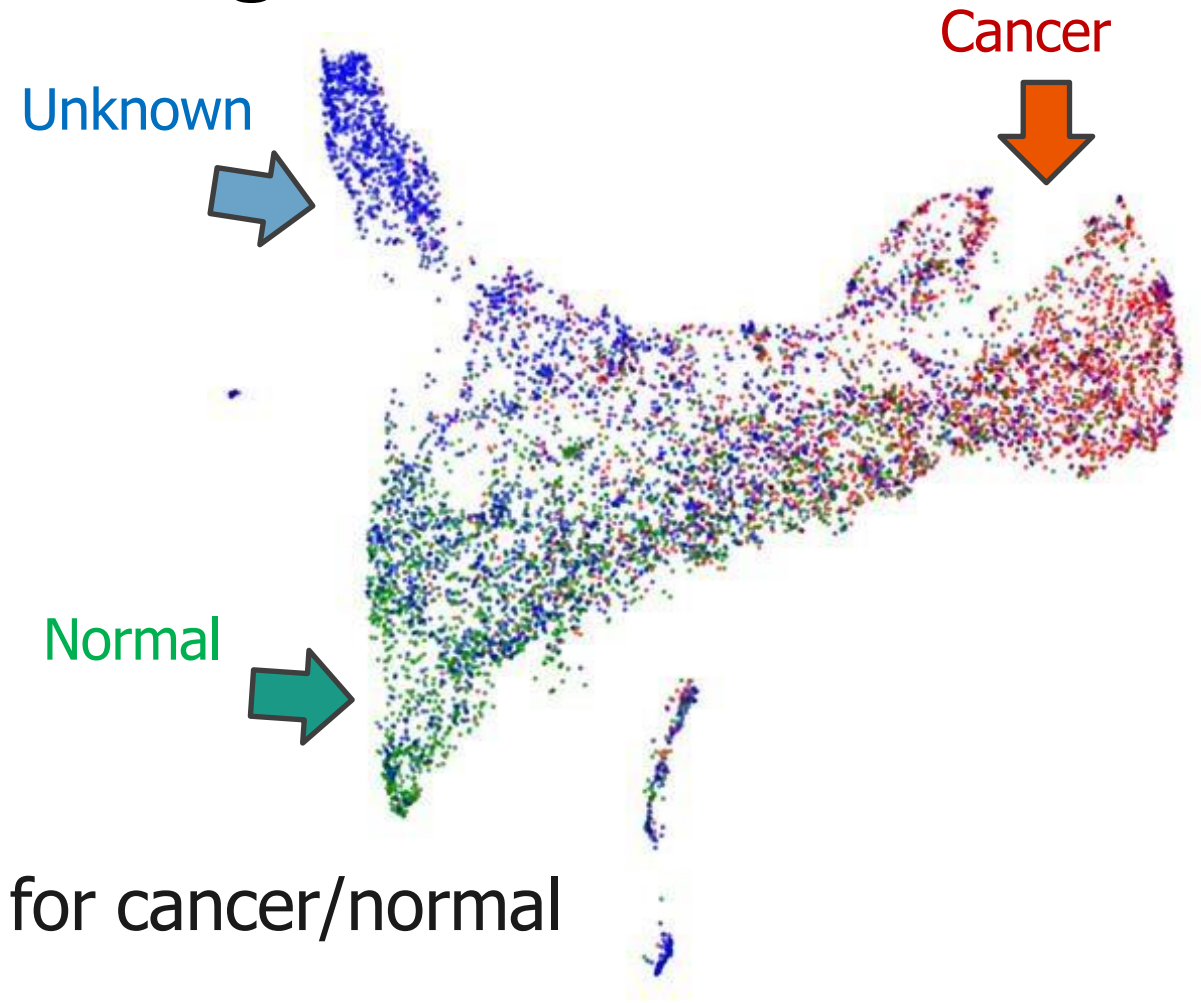
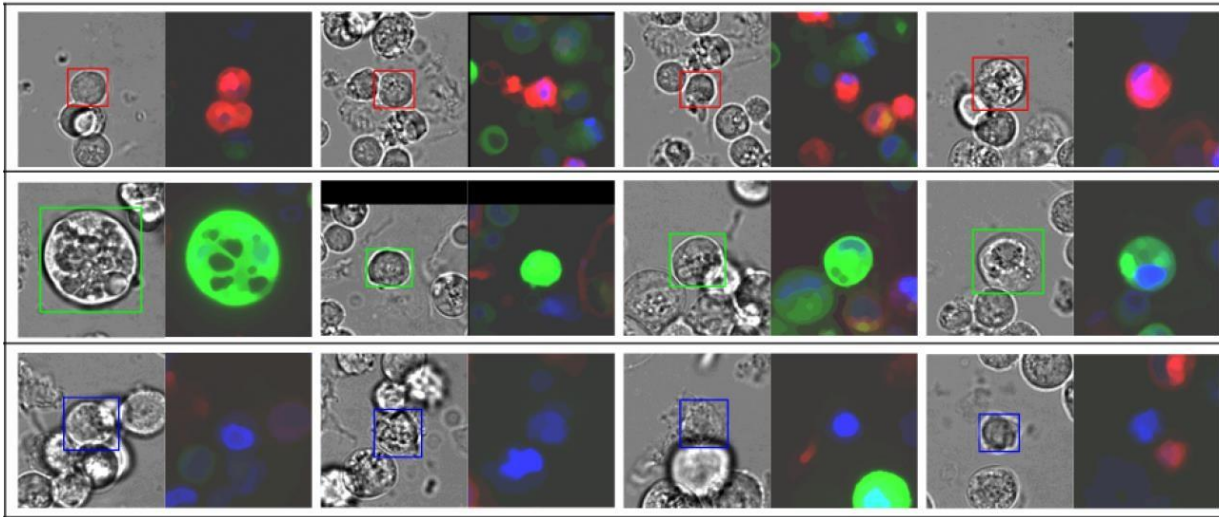
<https://github.com/zalandoresearch/fashion-mnist>



<https://pair-code.github.io/understanding-umap/> 8

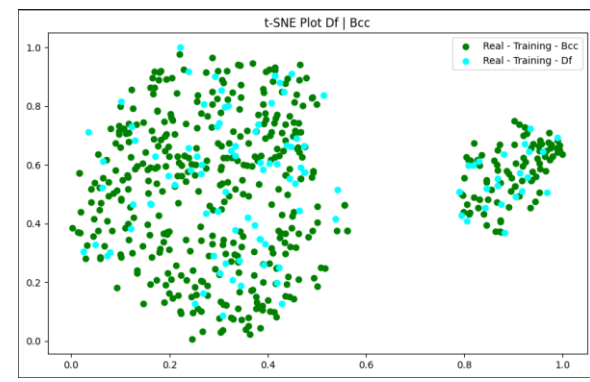
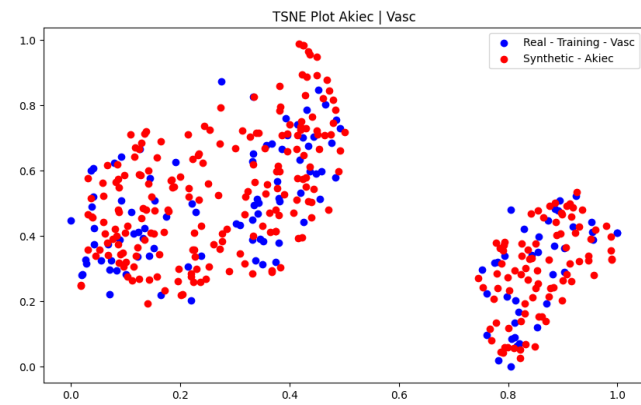
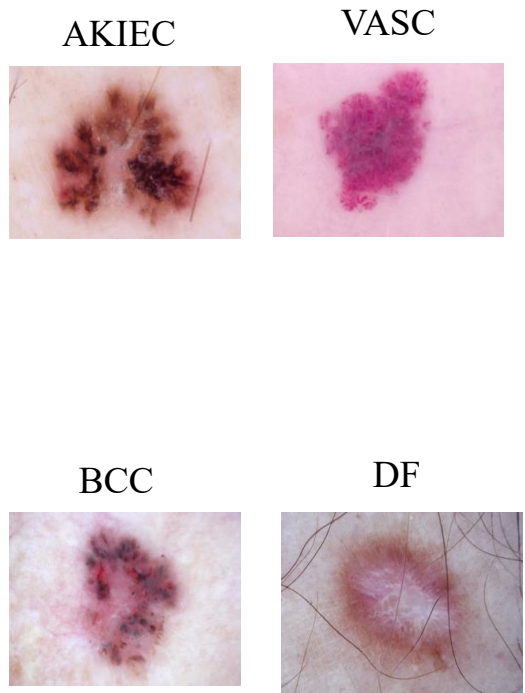


# 2D visualization for cell images

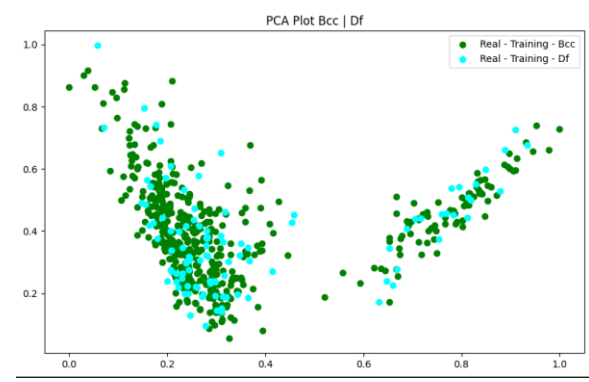
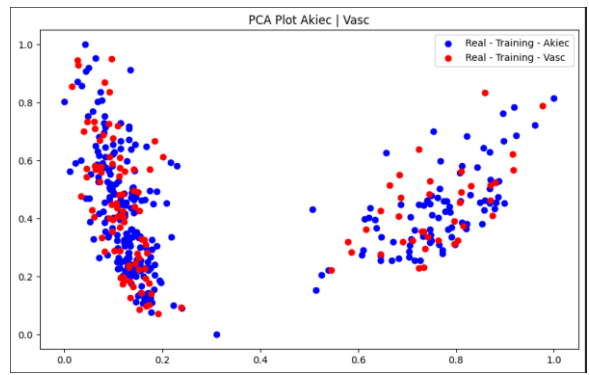


- Extracted from **deep learning model** for cancer/normal cell type classification

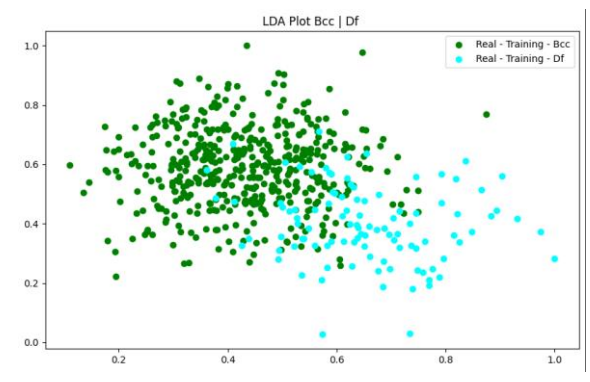
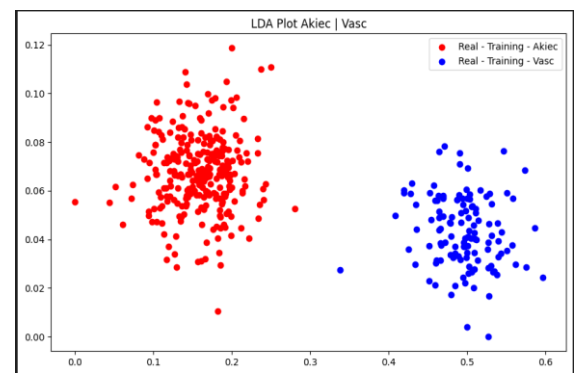
# Data visualization – t-SNE



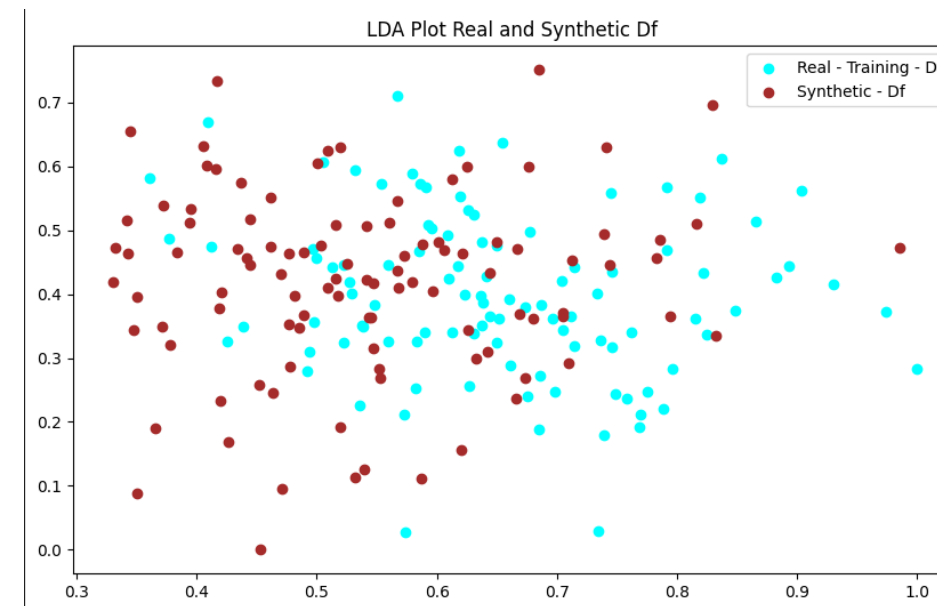
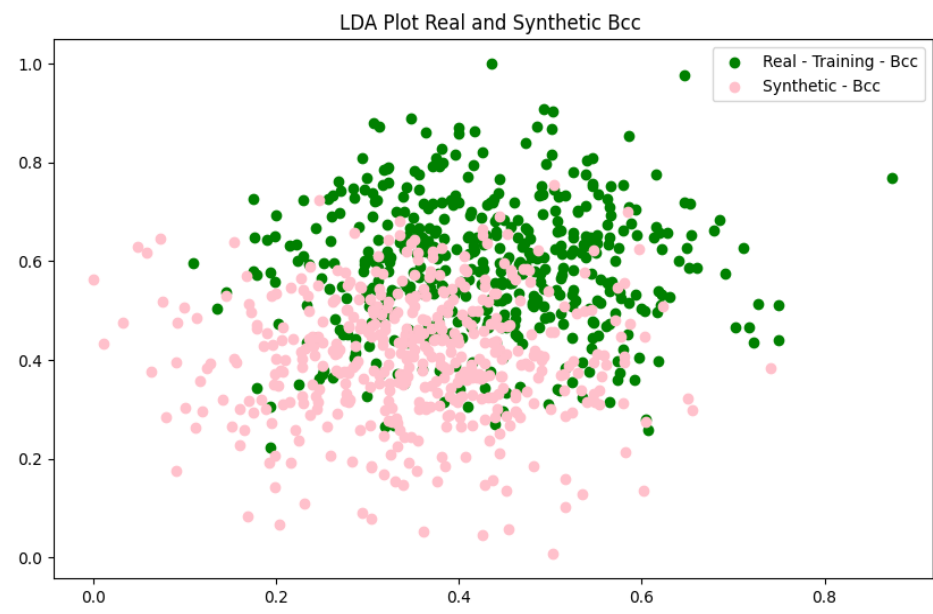
# PCA



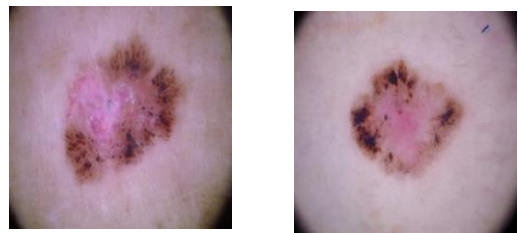
# LDA



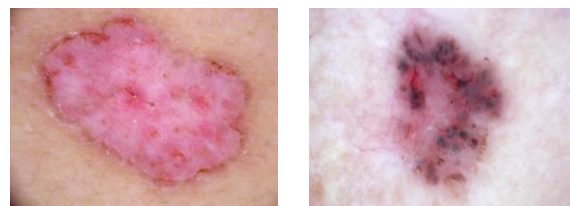
# Data visualization – LDA



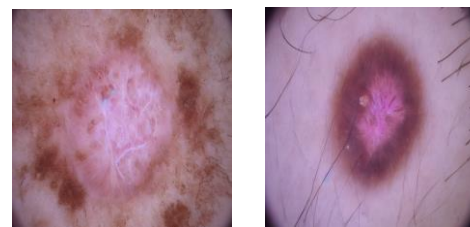
Synthetic images



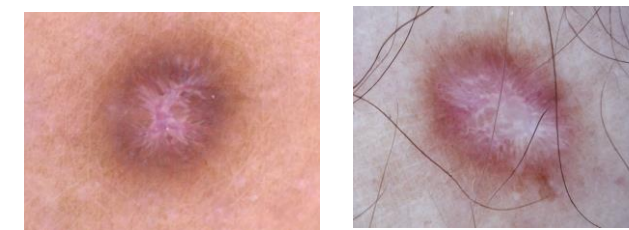
Real images



Synthetic images



Real images



Generating synthetic images for skin lesion classification

# Combating the curse of dimensionality

- Feature selection
  - Keep only “Good” features
- Feature transformation (Feature extraction)
  - Transform the original features into a smaller set of features

# Feature selection vs Feature transform (extraction)

- Keep original features
  - Useful for when the user wants to know which feature matters
    - Hard to select good features automatically
- New features (a combination of old features)
- Usually more powerful
  - Harder to interpret the model



# Feature selection

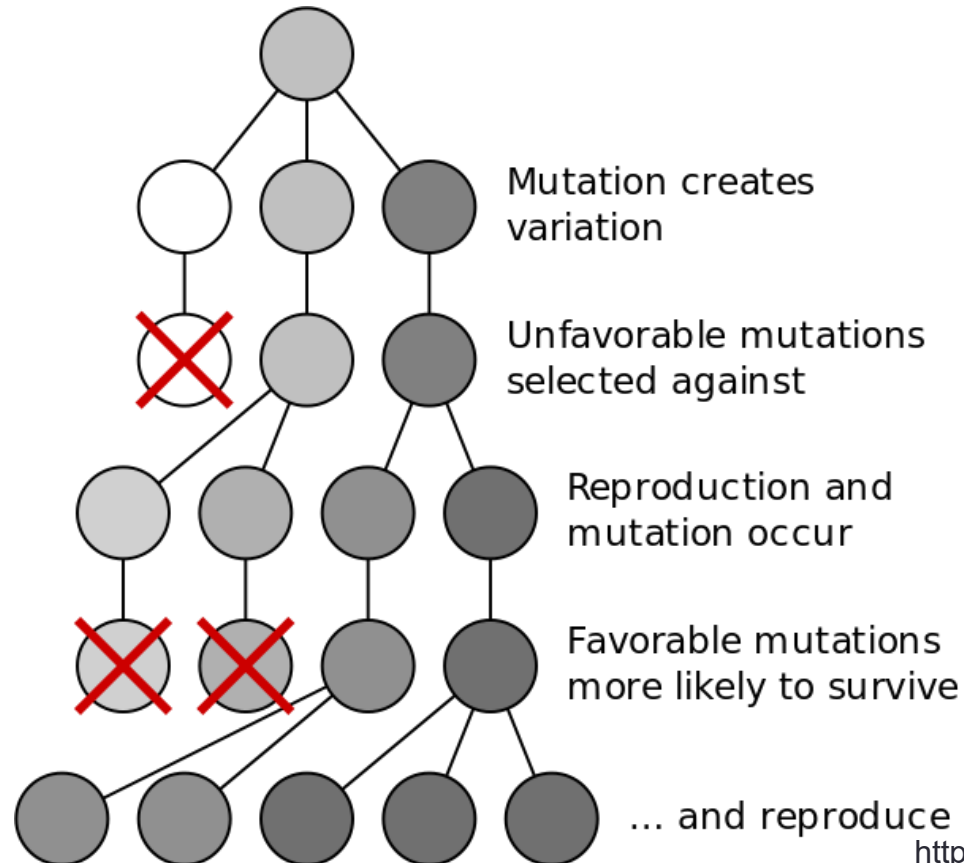
- Hackathon level (time limit days-a week)
  - Drop missing features
  - Low variance column
    - A feature that is a constant is useless. Tricky in practice
  - Forward or backward feature elimination
    - Greedy algorithm: create a simple classifier with  $n-1$  features,  $n$  times. Find which one has the best accuracy, drop that feature. Repeat.

# Feature selection

- Proper methods
  - Algorithm that handles high dimension well and do selection as a by product
  - Tree-based classifiers
    - Random forest
  - Adaboost
  - Genetic Algorithm

# Genetic Algorithm

- A method based inspired by natural selection
  - No theoretical guarantees but often work

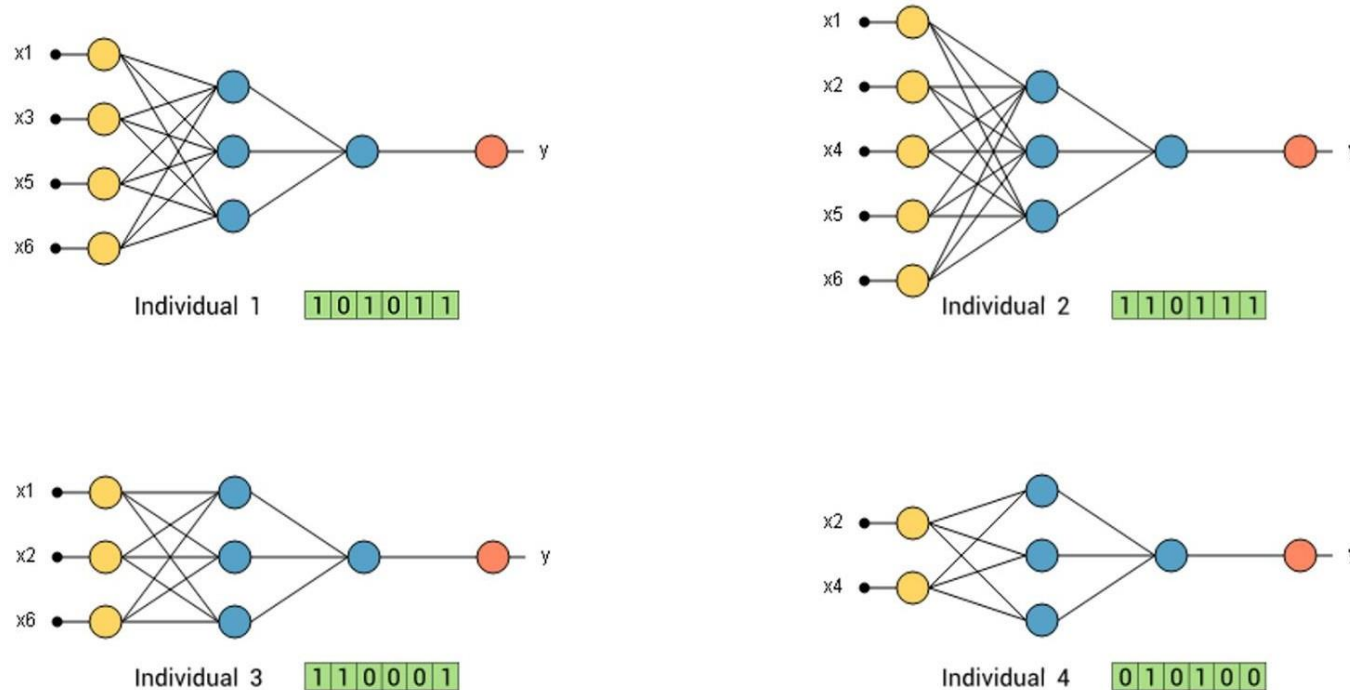


# Genetic Algorithm

- Initialization
  - Create N classifiers, each using different subset of features
- Selection process
  - Rank the N classifiers according to some criterion, kill the lower half
- Crossover
  - The remaining classifier breeds offsprings by selecting traits from the parents
- Mutation
  - The offsprings can have mutations by random in order to generate diversity
- Repeat till satisfied

# Initialization

- Create N classifiers
- Randomly select a subset of features to use





# Selection process

- Score the classifiers and kill the lower half (the amount to kill is also a parameter)

|                         | Selection error | Rank |
|-------------------------|-----------------|------|
| <del>Individual 1</del> | 0.9             | 1    |
| Individual 2            | 0.6             | 3    |
| <del>Individual 3</del> | 0.7             | 2    |
| Individual 4            | 0.5             | 4    |

# Crossover

- Breed offsprings by randomly select genes from parents

Individual 3      

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 1 |
|---|---|---|---|---|---|

Individual 4      

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|

---

Offspring 1      

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|

Offspring 2      

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|

Offspring 3      

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|

Offspring 4      

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|

# Mutation

- Offspring can mutate with some probability to introduce diversity
- Mutation rate is usually  $1/k$  where  $k$  is the number of features.
  - On average you mutate once per individual

Offspring1: Original    

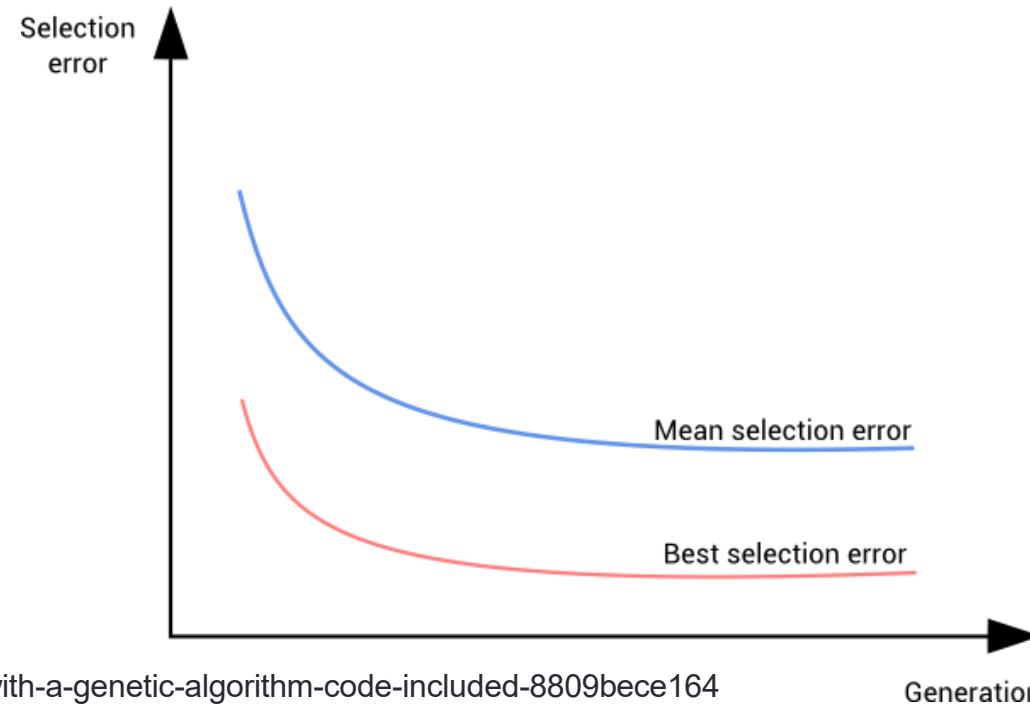
|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 1 |
|---|---|---|---|---|---|

Offspring1: Mutated    

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|

# Performance

- Usually performs well. The general population usually gets better (mean). The best performing (individual) also gets better after each generation
- Can be used to tune neural networks!



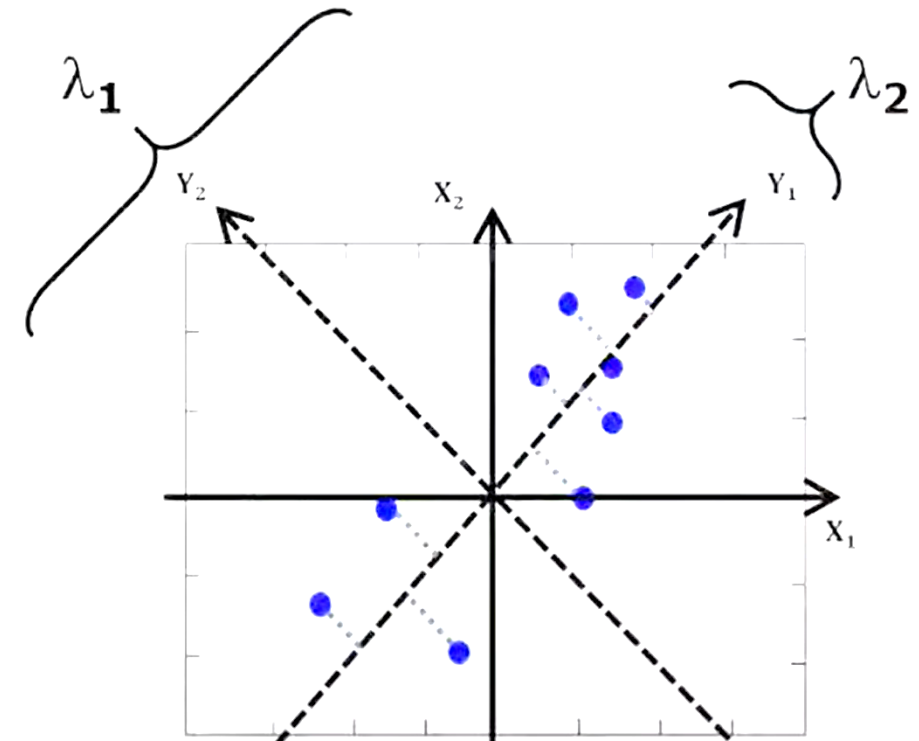
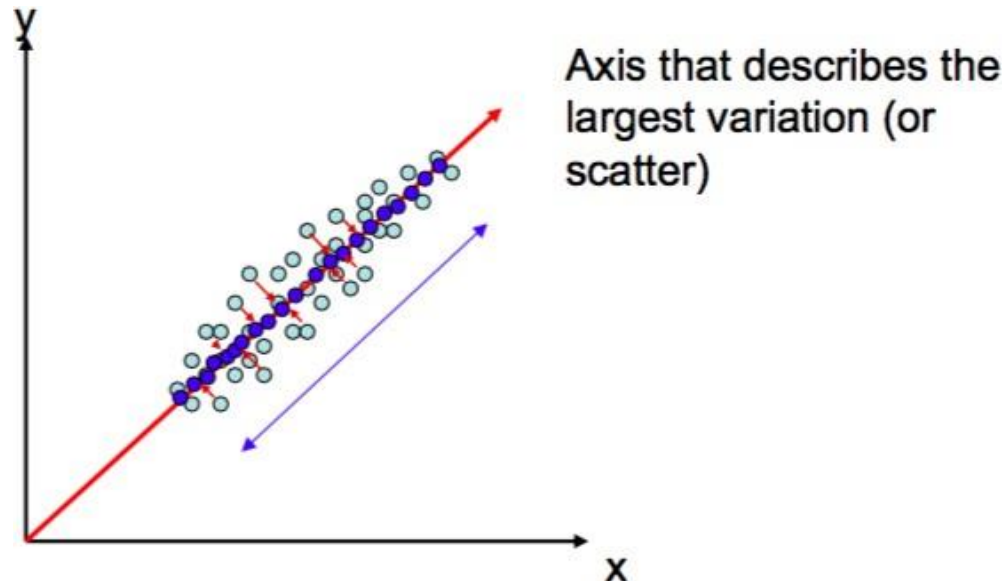
# Feature transformation (Extraction)

- Principal Component Analysis (PCA)\*\*
- Linear Discriminant Analysis (LDA)
- Multidimensional Scaling (MDS)
- Random Projections (R)
- t-distributed stochastic neighbor embedding (t-SNE) \*\*
- Uniform manifold approximation and projection (UMAP) \*\*

\*\* Let's demo

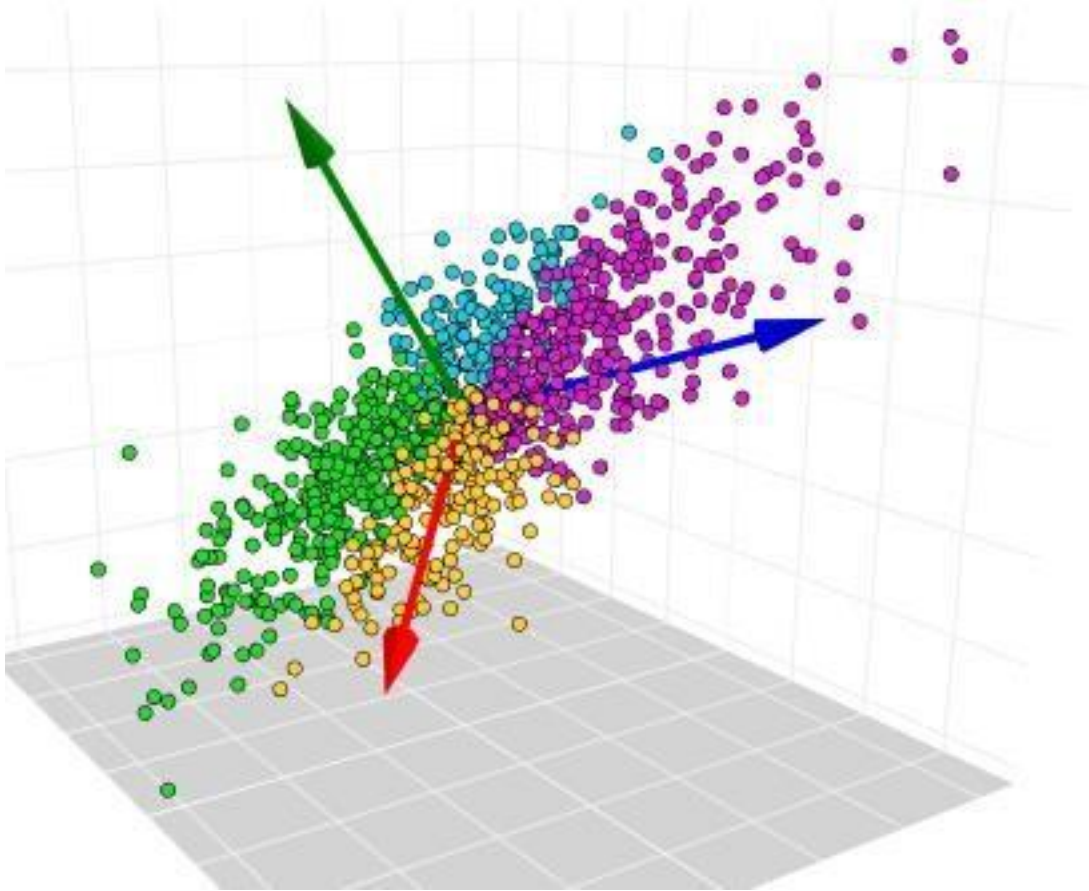
# What is PCA?

- We want to reduce the dimensionality but keep useful information
  - What is useful information? Variation
- We want to find a projection (a transformation) that
- describe maximum variation



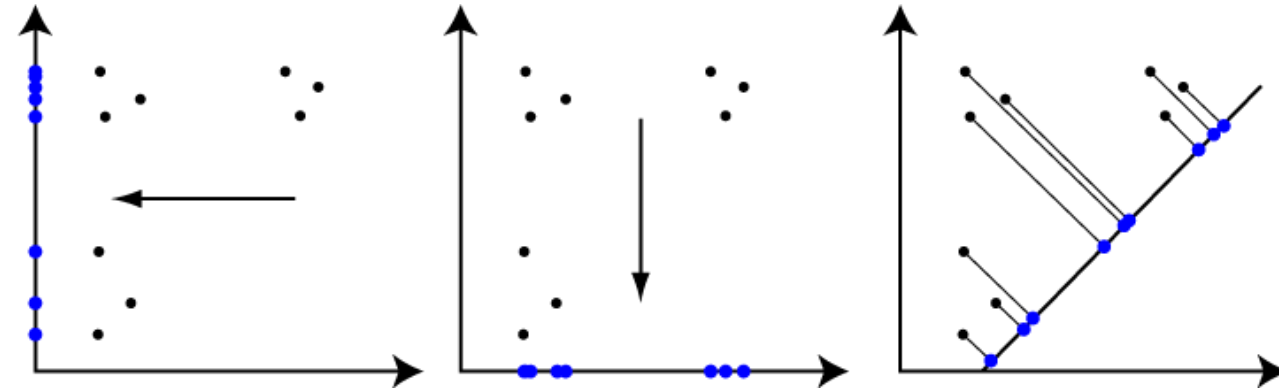


# Variance is information



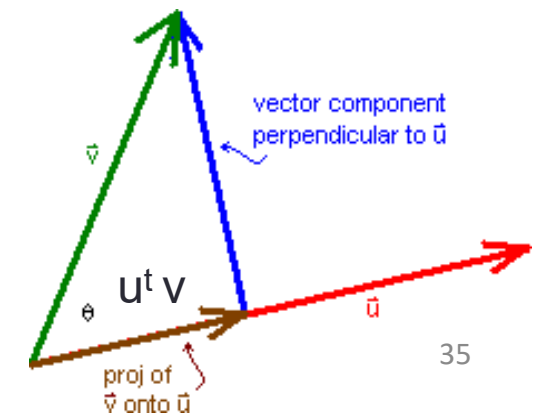
<https://towardsdatascience.com/principal-component-analysis-pca-explained-visually-with-zero-math-1cbf392b9e7d>

## Projection



<https://shapeofdata.wordpress.com/2013/04/16/visualization-and-projection/>

- High variances = more power to distinguish groups of data points



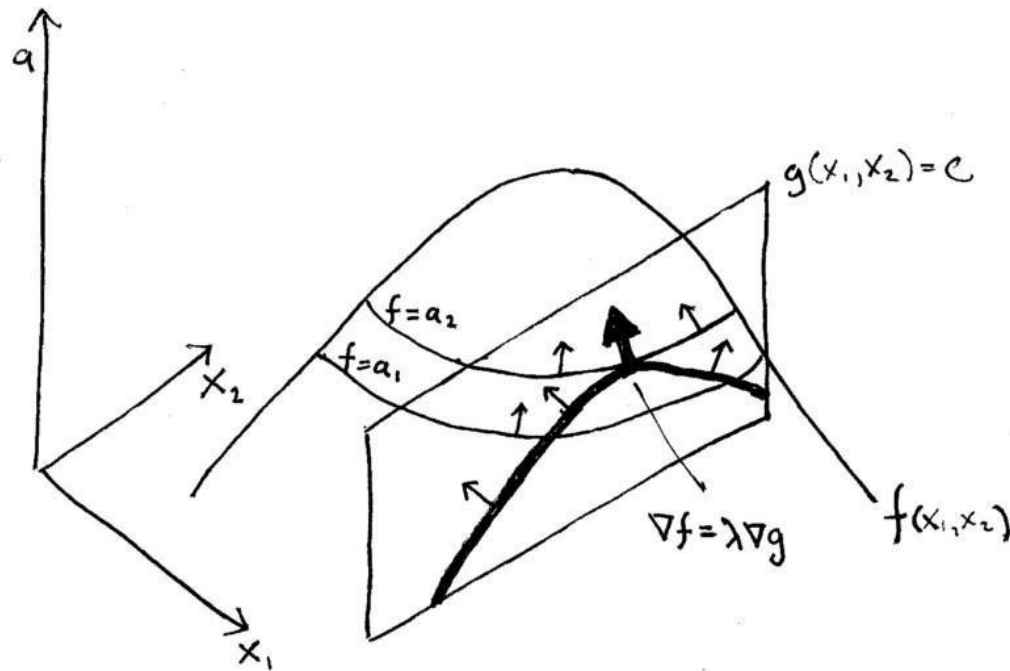
# Formulation

- Maximize the variance after projection ie
  - $\operatorname{argmax} \operatorname{Var}(\mathbf{w}^t \mathbf{x})$
- Subject to  $\mathbf{w}$  is a unit vector

- $\Sigma \mathbf{w} = \lambda \mathbf{w} \leftarrow$  eigenvector

# Lagrange Multiplier

<https://medium.com/@andrew.chamberlain/a-simple-explanation-of-why-lagrange-multipliers-works-253e2cdcbf74>



$$L(x_1, x_2, \lambda) = f(x_1, x_2) - \lambda(g(x_1, x_2) - c)$$

$$\nabla L = \begin{bmatrix} \frac{\partial f}{\partial x_1} - \lambda \frac{\partial g}{\partial x_1} \\ \frac{\partial f}{\partial x_2} - \lambda \frac{\partial g}{\partial x_2} \\ g(x_1, x_2) - c \end{bmatrix} = 0$$



# Covariance matrix

- Given a set of RVs,  $X_1 X_2 \dots X_n$
- The covariance matrix is a matrix which has the covariance of the  $i$  and  $j$  RV in position  $(i,j)$

$$\Sigma = \begin{bmatrix} E[(X_1 - \mu_1)(X_1 - \mu_1)] & E[(X_1 - \mu_1)(X_2 - \mu_2)] & \cdots & E[(X_1 - \mu_1)(X_n - \mu_n)] \\ E[(X_2 - \mu_2)(X_1 - \mu_1)] & E[(X_2 - \mu_2)(X_2 - \mu_2)] & \cdots & E[(X_2 - \mu_2)(X_n - \mu_n)] \\ \vdots & \vdots & \ddots & \vdots \\ E[(X_n - \mu_n)(X_1 - \mu_1)] & E[(X_n - \mu_n)(X_2 - \mu_2)] & \cdots & E[(X_n - \mu_n)(X_n - \mu_n)] \end{bmatrix}.$$

# Trace properties

- $\text{tr}(a) = a$
- $\text{tr}A = \text{tr}A^T$
- $\text{tr}(A+B) = \text{tr}A + \text{tr}B$
- $\text{tr}(aA) = a\text{tr}(A)$

$$\nabla_A \text{tr} AB = B^T \tag{1}$$

$$\nabla_{A^T} f(A) = (\nabla_A f(A))^T \tag{2}$$

$$\nabla_A \text{tr} ABA^T C = CAB + C^T AB^T \tag{3}$$

$$\nabla_{A^T} \text{tr} ABA^T C = B^T A^T C^T + BA^T C \tag{4}$$

# So we got to eigenvectors

- A  $d \times d$  covariance matrix has  $d$  eigenvectors/values pair.  
Do we use all of them?
- Which pair to use?

# Selecting eigenvectors

- Remember the variance of projected data is

$$\omega^T \Sigma \omega. \quad (1)$$

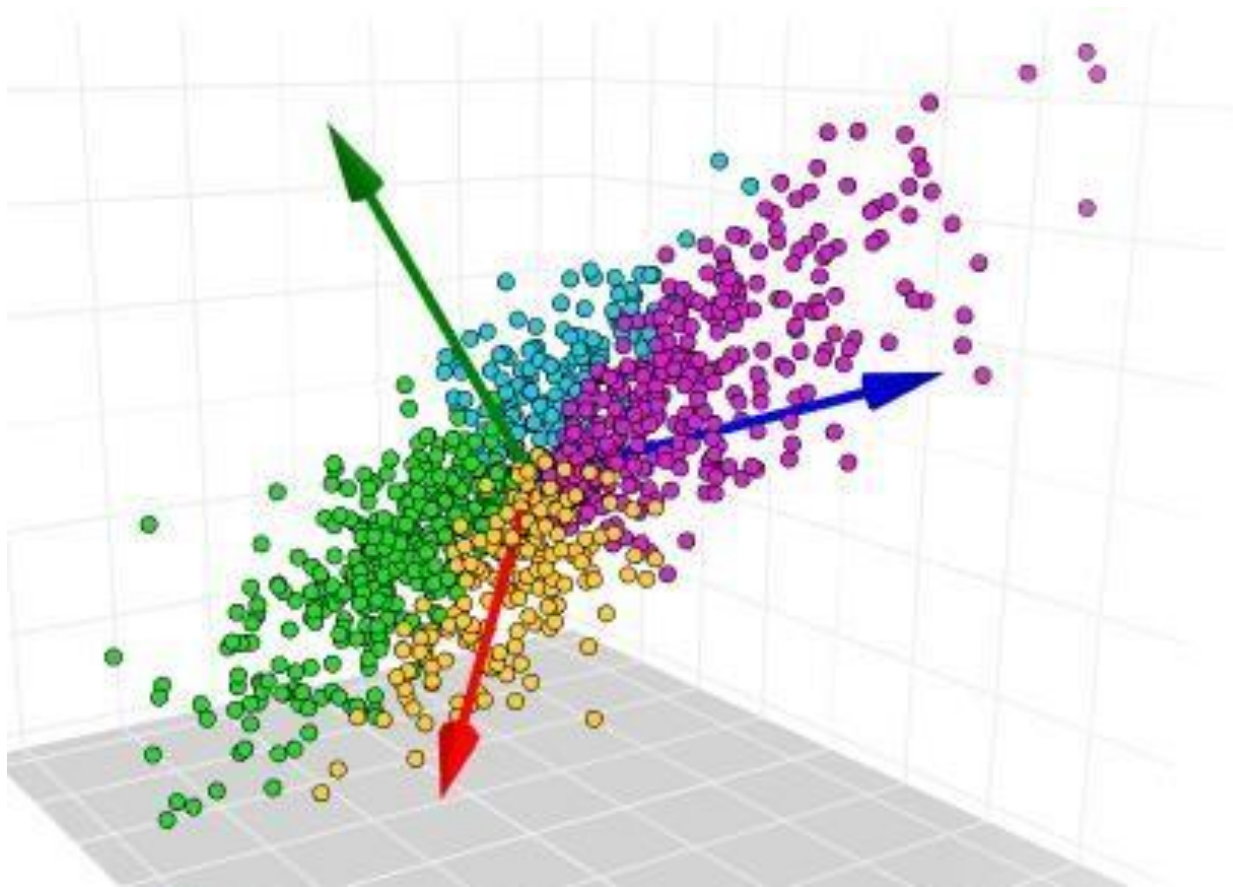
- And our solution yielded  $\Sigma \omega = \lambda \omega \quad (2)$

- Plug (2) in (1) and we get

$$\begin{aligned} \text{projected variance} &= \omega^T \Sigma \omega = \omega^T \lambda \omega \\ &= \lambda \omega^T \omega \quad (\text{remember } \|\omega\|=1) \\ &= \lambda \end{aligned}$$



# PCA



- The direction vector captures the variance corresponding to the eigenvalue
- So we want the higher eigenvalues
  - How many?

# Matrix rank

- A square  $d \times d$  matrix has full rank (e.g. rank  $d$ ) if the columns are linearly independent.
- The number of linearly independent columns is the rank of the matrix
- A covariance matrix of size  $d \times d$  will have at most  $N-1$  rank where  $N$  is the number of training samples
  - 640x640 images =  $\sim 400000$  dimensions
  - 1000 training images
  - The covariance matrix will be at most rank 999. The missing rank is because of the mean.

# PCA

- The direction vector captures the variance corresponding to the eigenvalue
- So we want the higher eigenvalues
- Take the eigenvalues with non-zero eigenvalues (at most  $N-1$  non-zero eigenvalues)

# Eigenfaces

Meanface



V1



V2



V3



V4



V5



V6



V7



V8



V9



V10



V11



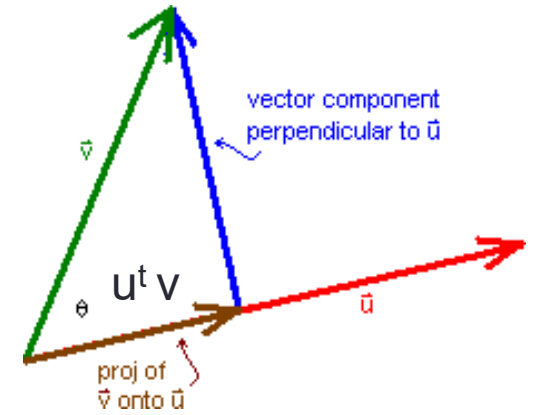
# Means

- In PCA, we model variance. (Variation around the mean)
- In our projection we need to remove the mean

$$\mathbf{p} = \mathbf{V}^T(\mathbf{x} - \mathbf{m})$$

- The mean is the mean of all your training data
- If we want to reconstruct the data we need to add back the mean

$$\mathbf{x} = \sum_{i=1}^N p_i \mathbf{v}_i + \mathbf{m} = p_1 \begin{bmatrix} | \\ \mathbf{v}_1 \\ | \end{bmatrix} + p_2 \begin{bmatrix} | \\ \mathbf{v}_2 \\ | \end{bmatrix} + \dots + p_n \begin{bmatrix} | \\ \mathbf{v}_n \\ | \end{bmatrix} + \mathbf{m} = \mathbf{V}\mathbf{p} + \mathbf{m}$$



# Basis decomposition

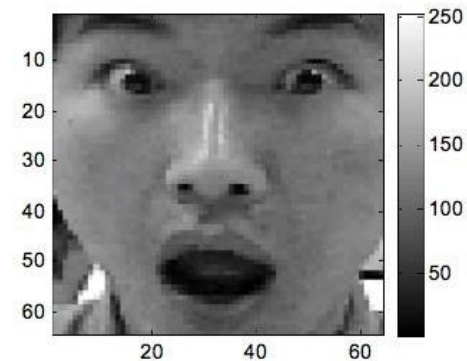
- Let's consider our projection  $w_i$  which is the eigenvectors to be a basis vector  $v_i$
- We can represent any vector as a sum of basis vectors as follows:

$$\mathbf{x} = \sum_{i=1}^N p_i \mathbf{v}_i = p_1 \begin{bmatrix} | \\ \mathbf{v}_1 \\ | \end{bmatrix} + p_2 \begin{bmatrix} | \\ \mathbf{v}_2 \\ | \end{bmatrix} + \dots + p_n \begin{bmatrix} | \\ \mathbf{v}_n \\ | \end{bmatrix} = \mathbf{V}\mathbf{p}$$

# Reconstruction with eigenfaces



reconstructed  
 $\tilde{\mathbf{X}}$



Original  
 $\mathbf{X}$

# How many eigenvectors?

- Select based on amount of variance explained
  - Sum of eigenvalues exceeds some percent of total
- Reconstruction error

$$\mathbf{x} = \sum_{i=1}^N p_i \mathbf{v}_i = p_1 \begin{bmatrix} | \\ \mathbf{v}_1 \\ | \end{bmatrix} + p_2 \begin{bmatrix} | \\ \mathbf{v}_2 \\ | \end{bmatrix} + \dots + p_n \begin{bmatrix} | \\ \mathbf{v}_n \\ | \end{bmatrix} = \mathbf{V}\mathbf{p}$$

- Select enough  $\mathbf{v}$  so that the difference between original  $\mathbf{x}$  and reconstructed  $\mathbf{x}$  is small

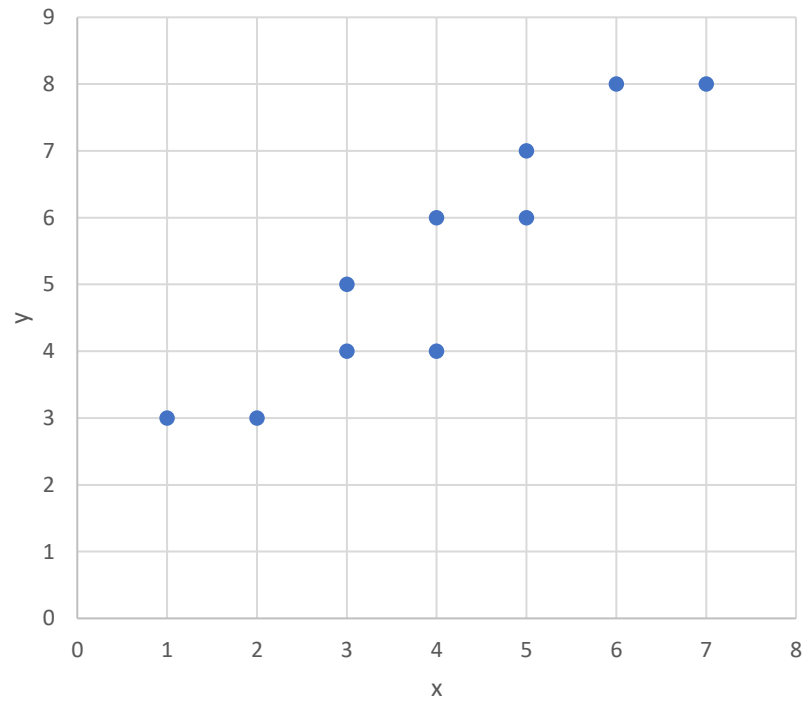


# PCA Summary



# PCA Example

| x             | y               |
|---------------|-----------------|
| 1             | 3               |
| 2             | 3               |
| 3             | 4               |
| 3             | 5               |
| 4             | 4               |
| 4             | 6               |
| 5             | 6               |
| 5             | 7               |
| 6             | 8               |
| 7             | 8               |
| $\bar{x} = 4$ | $\bar{y} = 5.4$ |



# PCA Example

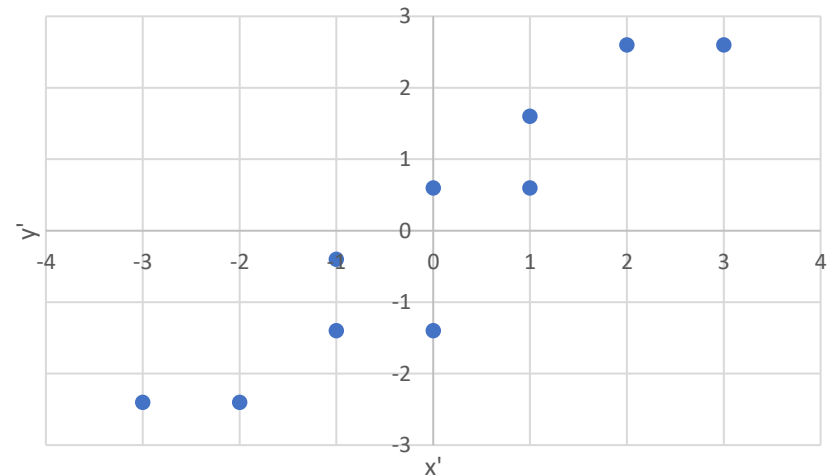
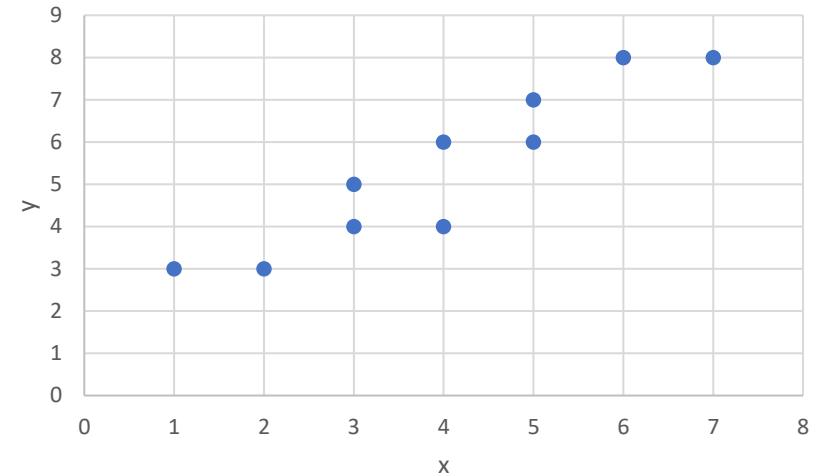
- Data Normalization

| x             | y               |
|---------------|-----------------|
| 1             | 3               |
| 2             | 3               |
| 3             | 4               |
| 3             | 5               |
| 4             | 4               |
| 4             | 6               |
| 5             | 6               |
| 5             | 7               |
| 6             | 8               |
| 7             | 8               |
| $\bar{x} = 4$ | $\bar{y} = 5.4$ |

Data  
Normalization



| x' | y'   |
|----|------|
| -3 | -2.4 |
| -2 | -2.4 |
| -1 | -1.4 |
| -1 | -0.4 |
| 0  | -1.4 |
| 0  | 0.6  |
| 1  | 0.6  |
| 1  | 1.6  |
| 2  | 2.6  |
| 3  | 2.6  |

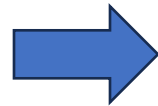


# PCA Example

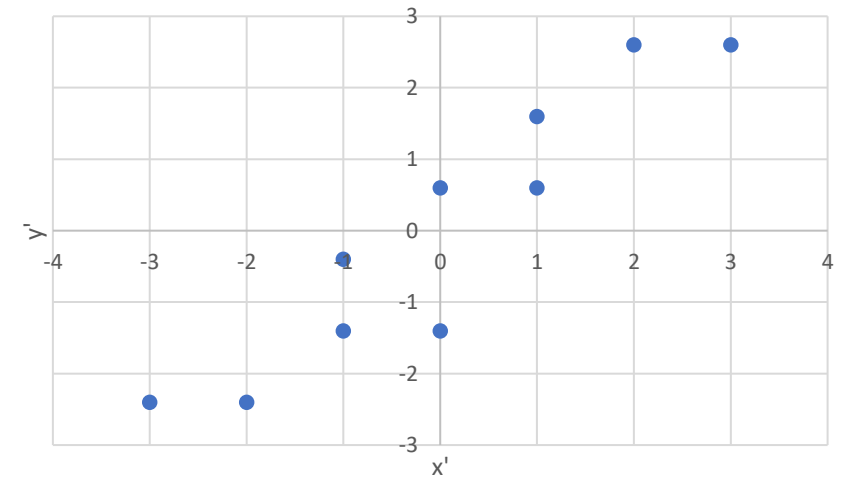
$$\text{Covariance Matrix } (C) = \frac{1}{M-1} AA^T$$

- Compute Covariance Matrix

| $A^T$ |      |
|-------|------|
| $x'$  | $y'$ |
| -3    | -2.4 |
| -2    | -2.4 |
| -1    | -1.4 |
| -1    | -0.4 |
| 0     | -1.4 |
| 0     | 0.6  |
| 1     | 0.6  |
| 1     | 1.6  |
| 2     | 2.6  |
| 3     | 2.6  |



| $C$  |      |
|------|------|
| 3.33 | 3.22 |
| 3.22 | 3.60 |



# PCA Example

$\lambda$  เป็นค่าเฉพาะของ  $A$  ก็ต่อเมื่อ  $(A - \lambda I_n)\vec{x} = \vec{0}_n$  มีผลเฉลยไม่ชัด  
ก็ต่อเมื่อ  $(A - \lambda I_n)$  ไม่มีเมทริกซ์ผกผัน  
ก็ต่อเมื่อ  $\det(A - \lambda I_n) = 0$

- Compute Eigenvalues and Eigenvectors

- $Cy_j = \lambda y_j \rightarrow (C - \lambda_j I)y_j = 0$

- $\begin{bmatrix} 3.33 - \lambda & 3.22 \\ 3.22 & 3.6 - \lambda \end{bmatrix} y_j = 0$

- $\det\left(\begin{bmatrix} 3.33 - \lambda & 3.22 \\ 3.22 & 3.6 - \lambda \end{bmatrix}\right) = 0$

- $(3.33 - \lambda)(3.6 - \lambda) - (3.22)(3.22) = 0 \rightarrow \lambda_1 = 0.242, \lambda_2 = 6.692$

$C$

|      |      |
|------|------|
| 3.33 | 3.22 |
| 3.22 | 3.60 |

# PCA Example

$\lambda$  เป็นค่าเฉพาะของ  $A$  ก็ต่อเมื่อ  $(A - \lambda I_n)\vec{x} = \vec{0}_n$  มีผลเฉลยไม่ซัด  
 ก็ต่อเมื่อ  $(A - \lambda I_n)$  ไม่มีเมทริกซ์ผกผัน  
 ก็ต่อเมื่อ  $\det(A - \lambda I_n) = 0$

- Compute Eigenvalues and Eigenvectors

- *Case* :  $\lambda_1 = 0.242$

- $\begin{bmatrix} 3.33 - 0.242 & 3.22 \\ 3.22 & 3.6 - 0.242 \end{bmatrix} y_j = \begin{bmatrix} 3.33 - 0.242 & 3.22 \\ 3.22 & 3.6 - 0.242 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = 0$

- $\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} -0.722 \\ 0.692 \end{bmatrix}$

## Eigenvectors

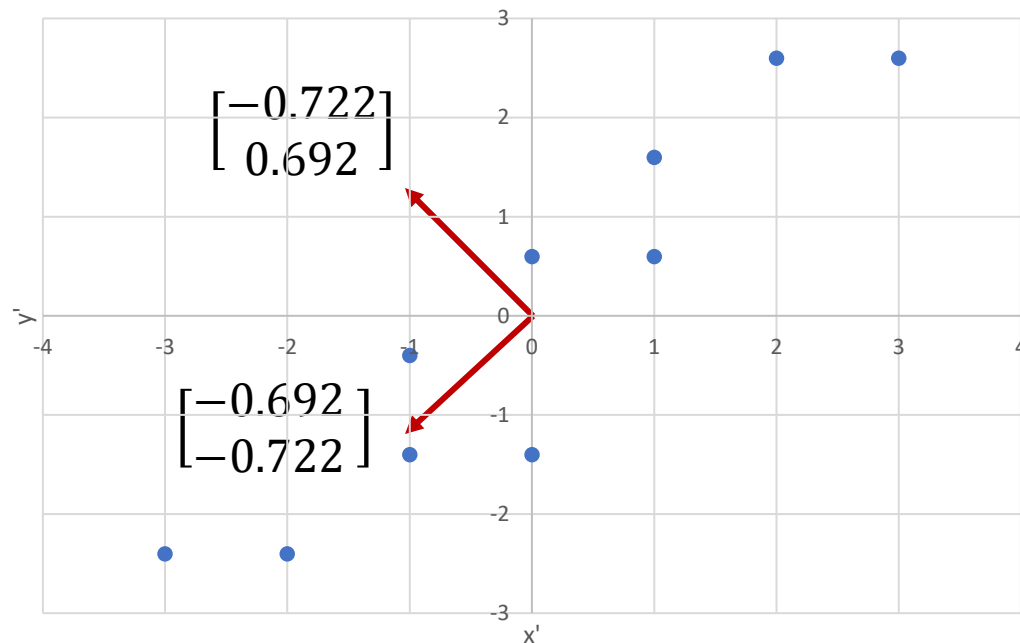
|        |        |
|--------|--------|
| -0.722 | -0.692 |
| 0.692  | -0.722 |
| $y_1$  | $y_2$  |

## Eigenvalues

$$\lambda_1 = 0.242, \lambda_2 = 6.692$$

# PCA Example

- Rotate axis and eliminate dimension

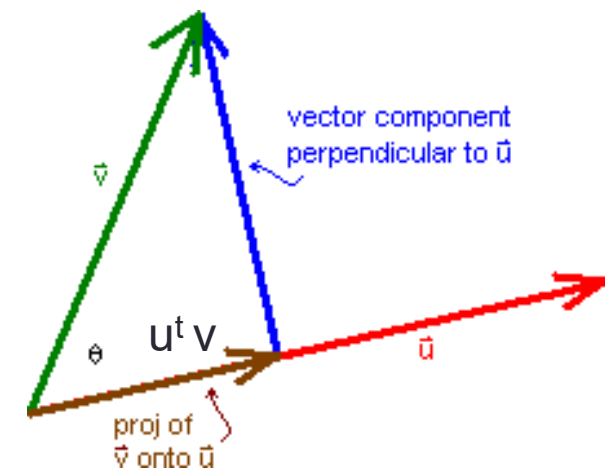


## Eigenvalues

$$\lambda_1 = 0.242, \lambda_2 = 6.692$$

## Eigenvectors

|          |          |
|----------|----------|
| $-0.692$ | $-0.722$ |
| $-0.722$ | $0.692$  |
| $y_2$    | $y_1$    |



# PCA Example

- Rotate axis and eliminate dimension

| $A^T$ |      |                                                                                                     | $(A')^T$ |        |
|-------|------|-----------------------------------------------------------------------------------------------------|----------|--------|
| $x'$  | $y'$ |                                                                                                     | $x'$     | $y'$   |
| -3    | -2.4 | $A' = y^T A$<br> | 3.809    | 0.505  |
| -2    | -2.4 |                                                                                                     | 3.117    | -0.217 |
| -1    | -1.4 |                                                                                                     | 1.703    | -0.247 |
| -1    | -0.4 |                                                                                                     | 0.981    | 0.445  |
| 0     | -1.4 |                                                                                                     | 1.011    | -0.969 |
| 0     | 0.6  |                                                                                                     | -0.433   | 0.415  |
| 1     | 0.6  |                                                                                                     | -1.125   | -0.307 |
| 1     | 1.6  |                                                                                                     | -1.847   | 0.385  |
| 2     | 2.6  |                                                                                                     | -3.261   | 0.355  |
| 3     | 2.6  |                                                                                                     | -3.953   | -0.367 |



# PCA Example

- Rotate axis and eliminate dimension (Inverse)

| x'     |
|--------|
| 3.809  |
| 3.117  |
| 1.703  |
| 0.981  |
| 1.011  |
| -0.433 |
| -1.125 |
| -1.847 |
| -3.261 |
| -3.953 |



Eigenvectors (y<sub>2</sub>)

|        |        |
|--------|--------|
| -0.692 | -0.722 |
|--------|--------|

=

1 x 2

| x'       | y'       |
|----------|----------|
| -2.63583 | -2.7501  |
| -2.15696 | -2.25047 |
| -1.17848 | -1.22957 |
| -0.67885 | -0.70828 |
| -0.69961 | -0.72994 |
| 0.299636 | 0.312626 |
| 0.7785   | 0.81225  |
| 1.278124 | 1.333534 |
| 2.256612 | 2.354442 |
| 2.735476 | 2.854066 |

≈

| x' | y'   |
|----|------|
| -3 | -2.4 |
| -2 | -2.4 |
| -1 | -1.4 |
| -1 | -0.4 |
| 0  | -1.4 |
| 0  | 0.6  |
| 1  | 0.6  |
| 1  | 1.6  |
| 2  | 2.6  |
| 3  | 2.6  |

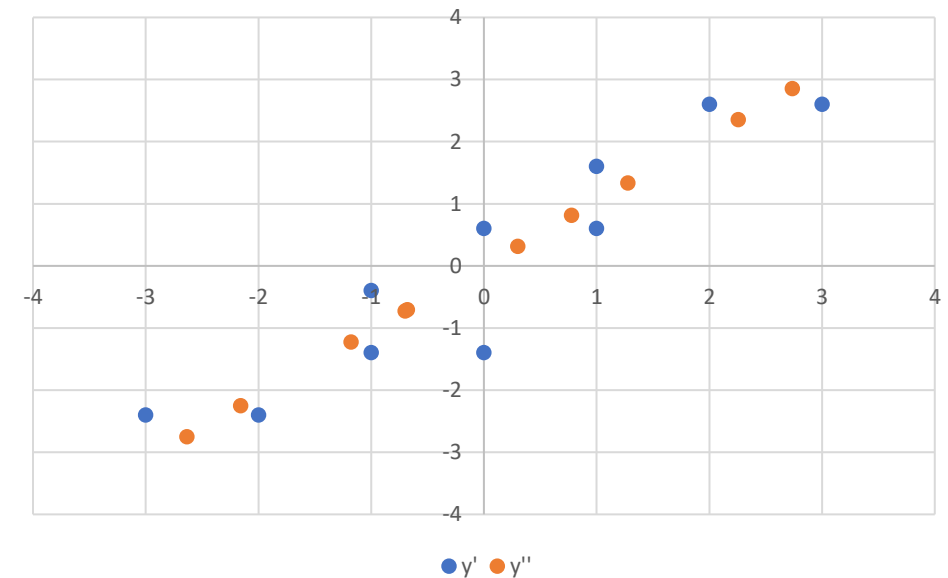
M x 1

M x 2

# PCA Example

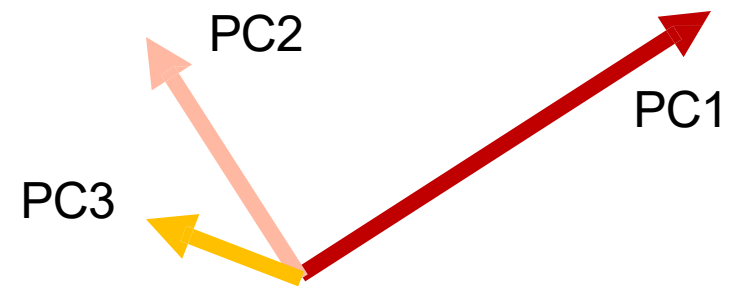
- Rotate axis and eliminate dimension (Inverse)

| $x'$ | $y'$ | PCA<br>➔ | $x'$   | Inverse<br>➔ | $x'$     | $y'$     |
|------|------|----------|--------|--------------|----------|----------|
| -3   | -2.4 |          | 3.809  |              | -2.63583 | -2.7501  |
| -2   | -2.4 |          | 3.117  |              | -2.15696 | -2.25047 |
| -1   | -1.4 |          | 1.703  |              | -1.17848 | -1.22957 |
| -1   | -0.4 |          | 0.981  |              | -0.67885 | -0.70828 |
| 0    | -1.4 |          | 1.011  |              | -0.69961 | -0.72994 |
| 0    | 0.6  |          | -0.433 |              | 0.299636 | 0.312626 |
| 1    | 0.6  |          | -1.125 |              | 0.7785   | 0.81225  |
| 1    | 1.6  |          | -1.847 |              | 1.278124 | 1.333534 |
| 2    | 2.6  |          | -3.261 |              | 2.256612 | 2.354442 |
| 3    | 2.6  |          | -3.953 |              | 2.735476 | 2.854066 |

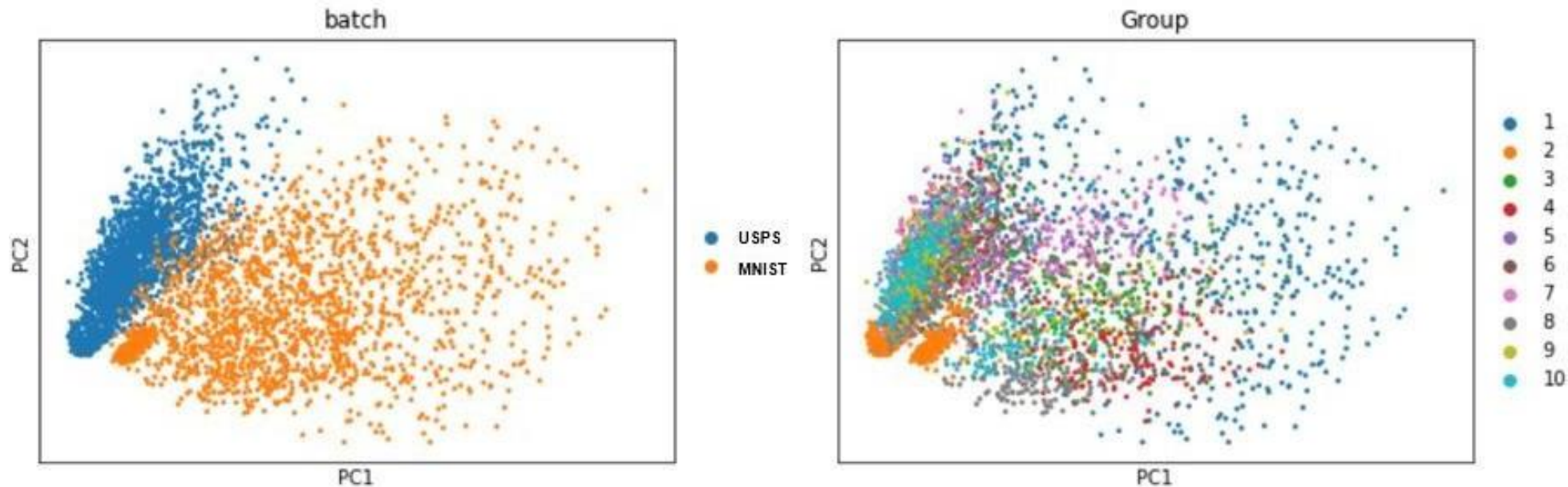


# Interpretations of PCA algorithm

- Orthogonal linear transformation
  - New axes are **rotations** of the original axes
  - $PC1 = w_1x_1 + w_2x_2 + \dots + w_nx_n$
  - $PC2 = v_1x_1 + v_2x_2 + \dots + v_nx_n$
  - $w_i$ 's and  $v_i$ 's are called **loadings**

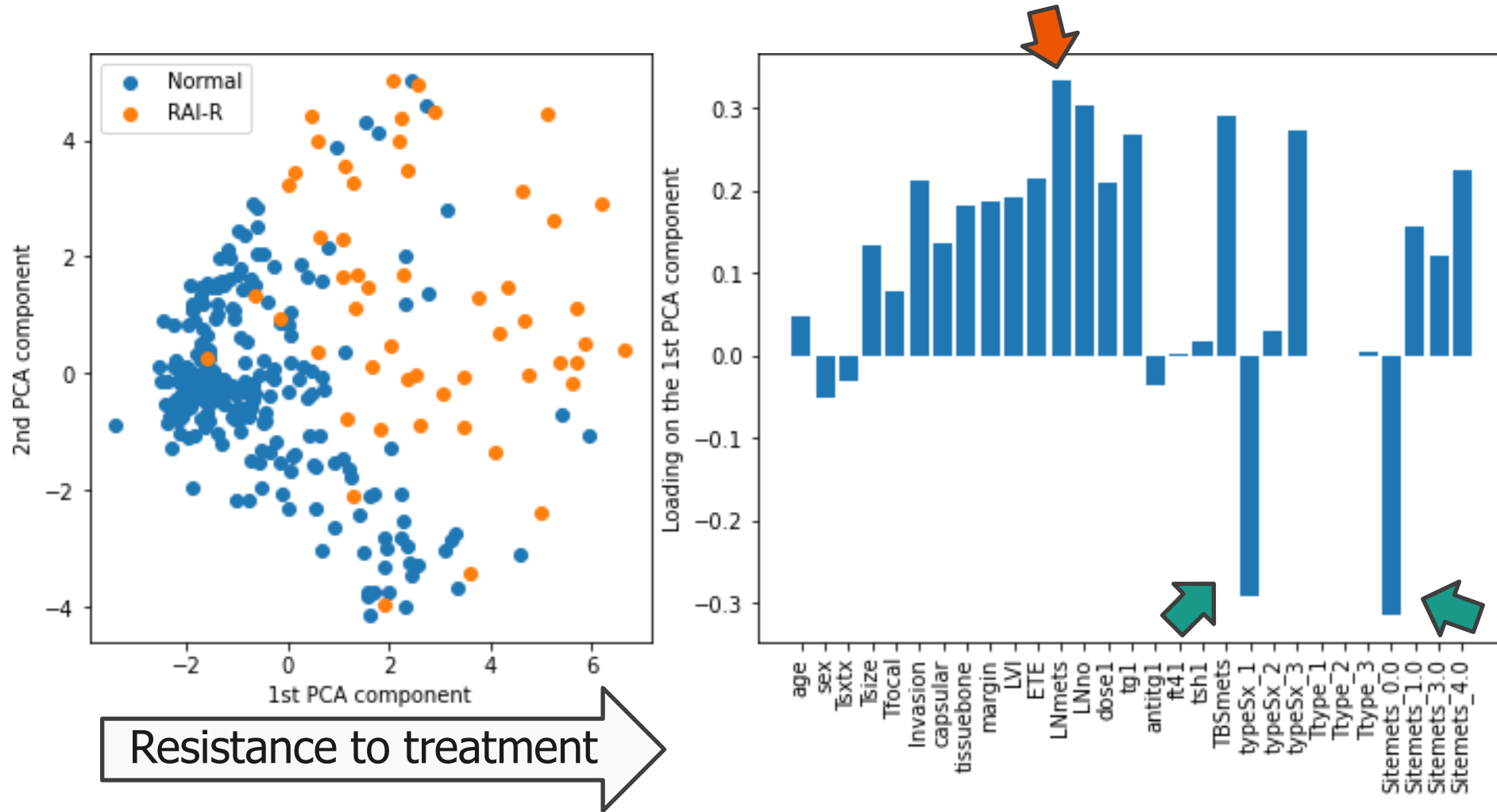


# Interpretation of PCA result

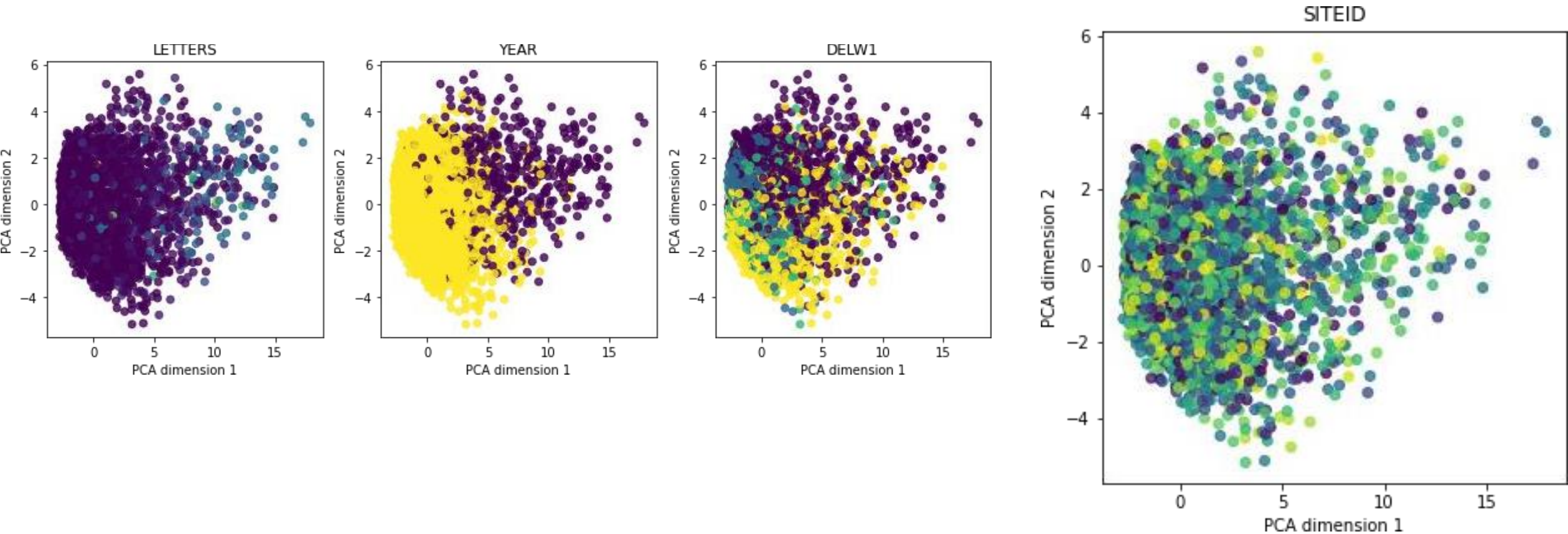


- PC1 captures the variance between data sources
- PC2 somewhat captures the variance between digit identity

# Interpreting loadings on individual PC

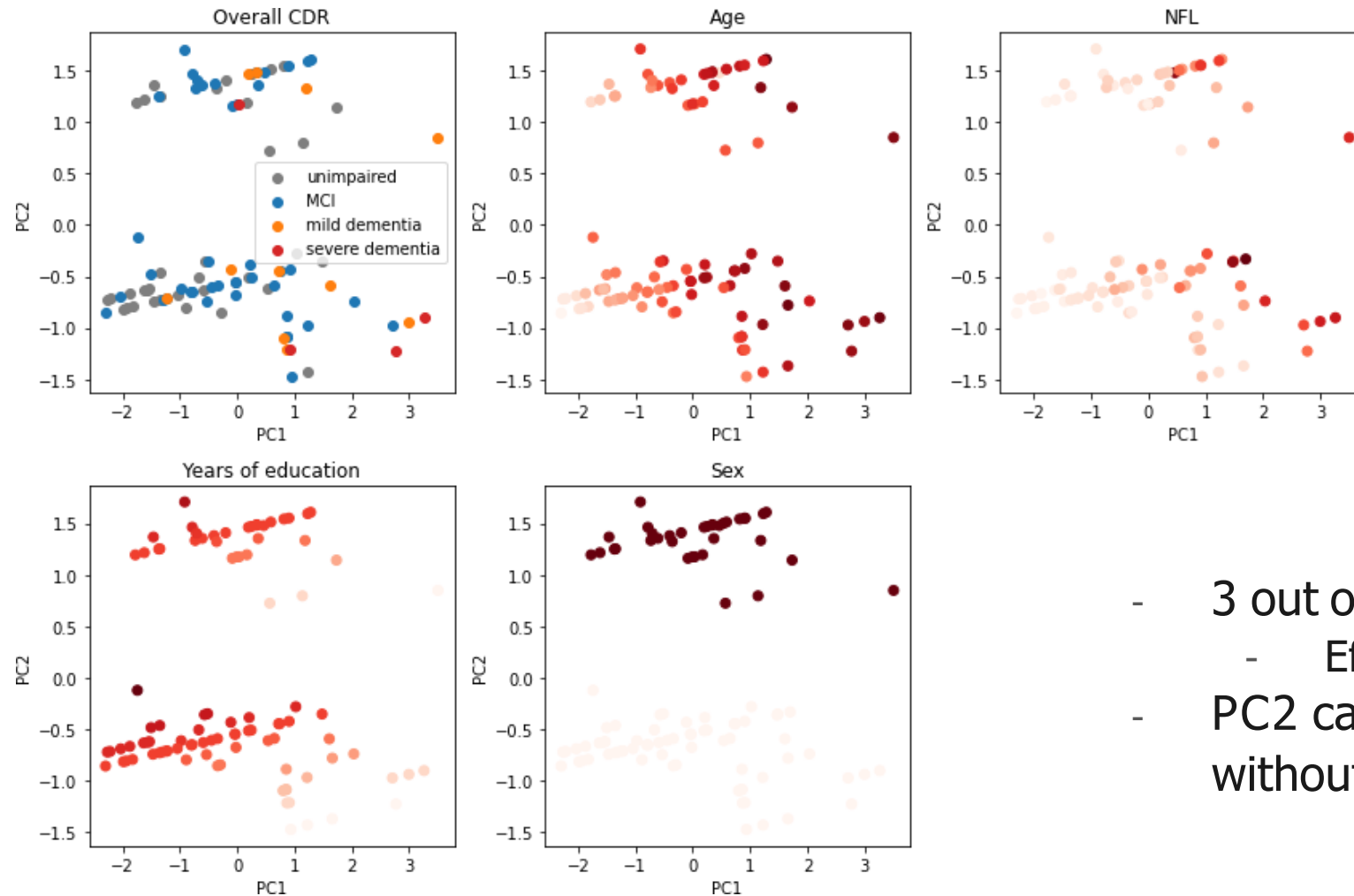


# Exploring PCA results



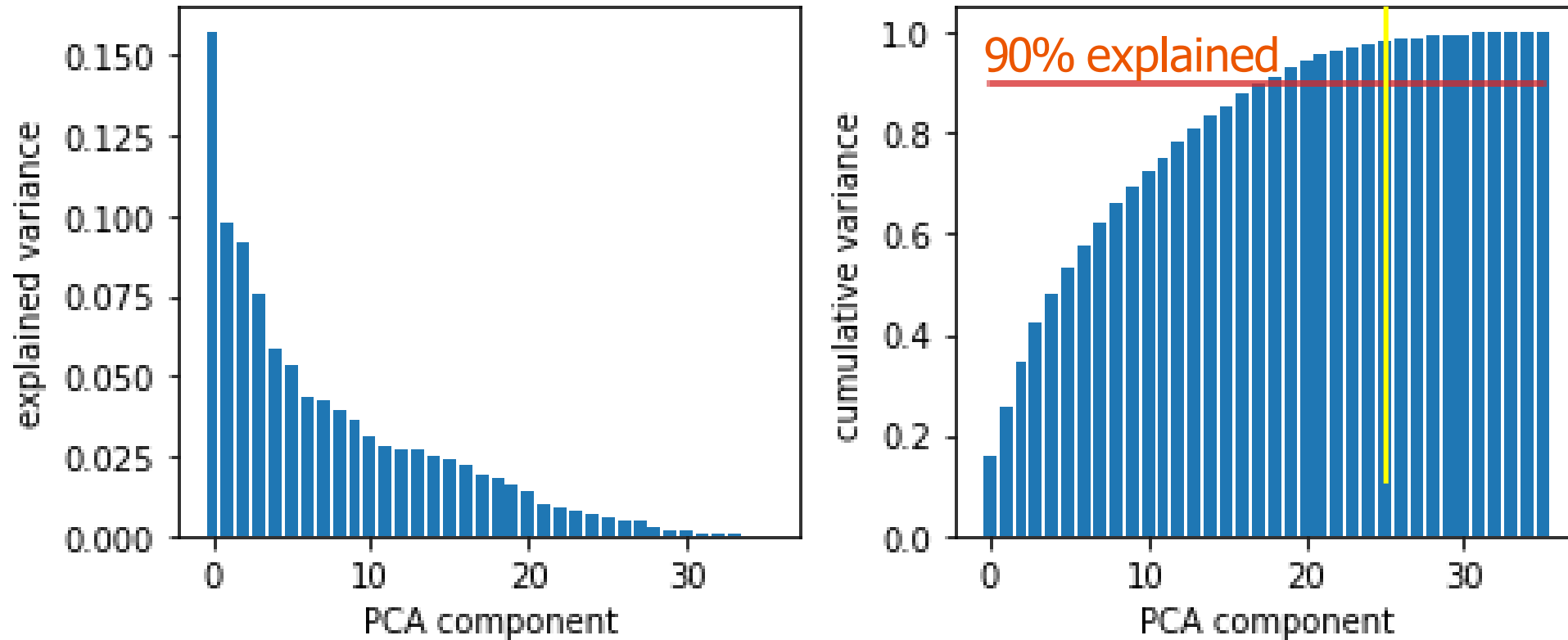
- Color by feature values to understand how PCA group data points
- Color by potential confounding factors

# Be careful of correlated features



- 3 out of 4 features are correlated
  - Effective dimension is 2
- PC2 captures the gender difference without meaningful interpretation

# PCA for dimensionality reduction



- By default, PCA retains the number of dimensions
- We can **select only the first  $k$  PC** for downstream analyses



# PCA in Python

## `sklearn.decomposition.PCA`

```
class sklearn.decomposition.PCA(n_components=None, *, copy=True, whiten=False, svd_solver='auto', tol=0.0,
iterated_power='auto', n_oversamples=10, power_iteration_normalizer='auto', random_state=None)
```

- **n\_components** = number of directions/axes to produce
  - default = min(sample, feature)
- Input data will be **automatically centered** (set mean = 0 for each feature) **but not scaled** (set variance = 1 for each feature)
- **random\_state** controls the behavior of algorithm that have some randomness
  - Always set to a fixed value for reproducibility

# PCA usage

```
from sklearn.decomposition import PCA
```

```
pca = PCA(random_state = 25)  
pca.fit(input_data)
```

```
## a fitted PCA can be used to transform any new  
data input_embed = pca.transform(input_data)  
output_embed = pca.transform(output_data)
```

```
## you can fit & transform directly  
new_embed = pca.fit_transform(new_data)
```

# PCA output

## number of output directions/axes

`pca.n_components_`

## array of variance explained by each axis (from high to low, sum to one)

`pca.explained_variance_ratio_`

## 2D array of the loading ( $w_i$ 's) of each feature on each  
direction/axis ## dimension = # direction x # feature

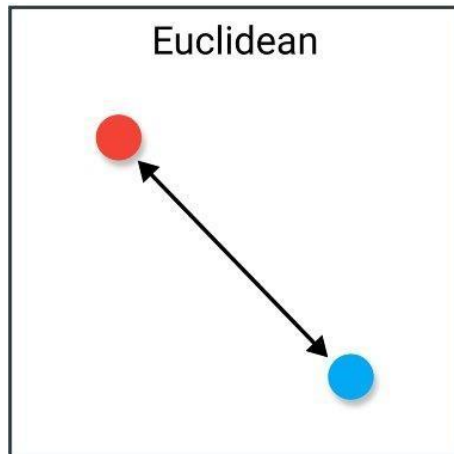
`pca.components_`

## the loading ( $w_i$ 's) of each feature on the 3<sup>rd</sup>

direction/axis `pca.components_[2]`

# Pros and cons of PCA

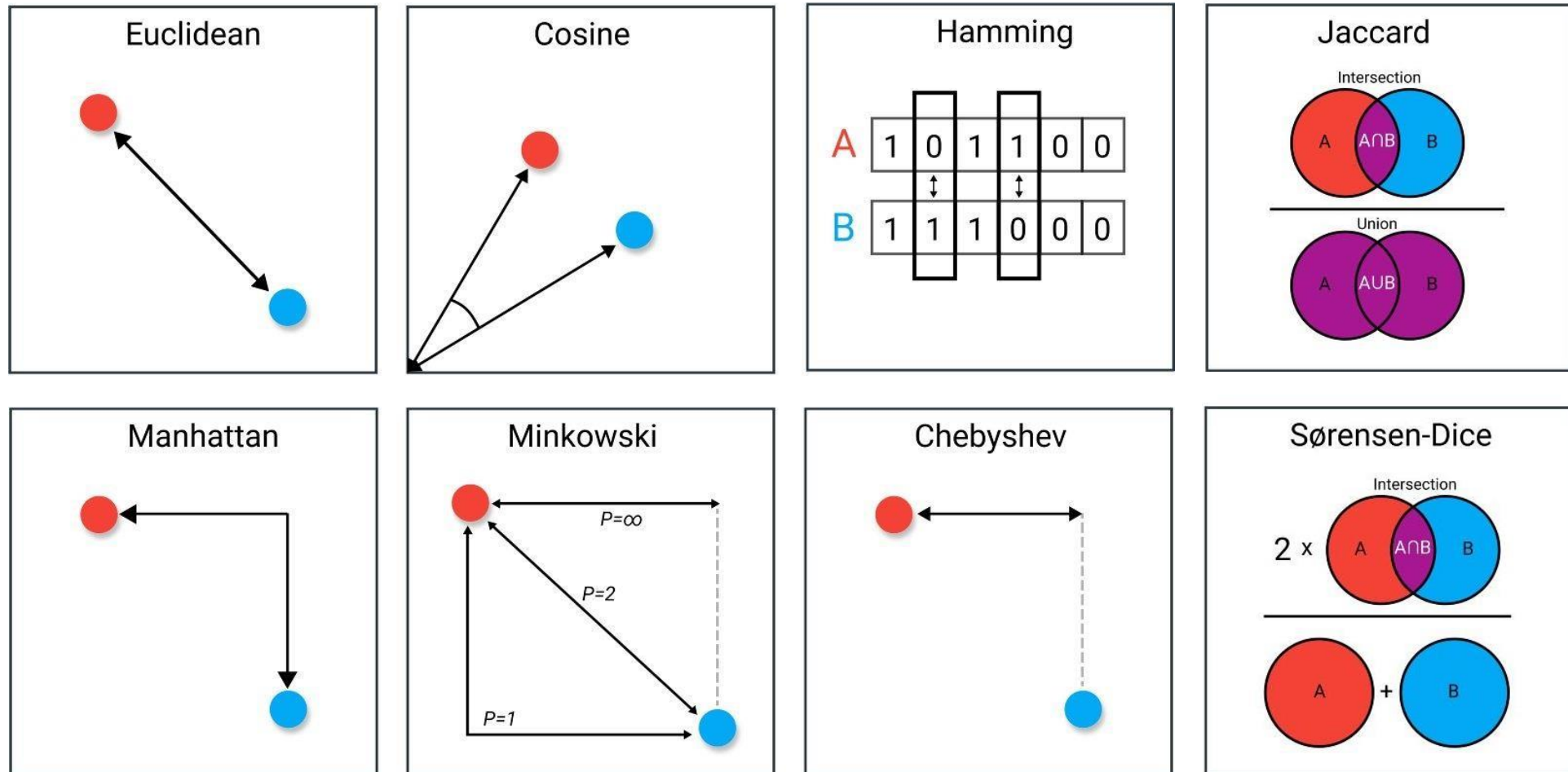
- Each PC can be interpreted from the **loadings**
- **Highly correlated features tend** to be grouped into the same PC
- PCA is a good initial dimensionality reduction step
- PCA strictly **preserves Euclidean distance**
  - But some datasets require different distance metric!



<https://towardsdatascience.com/9-distance-measures-in-data-science-918109d069fa>

# Multidimensional Scaling (MDS)

## Distances

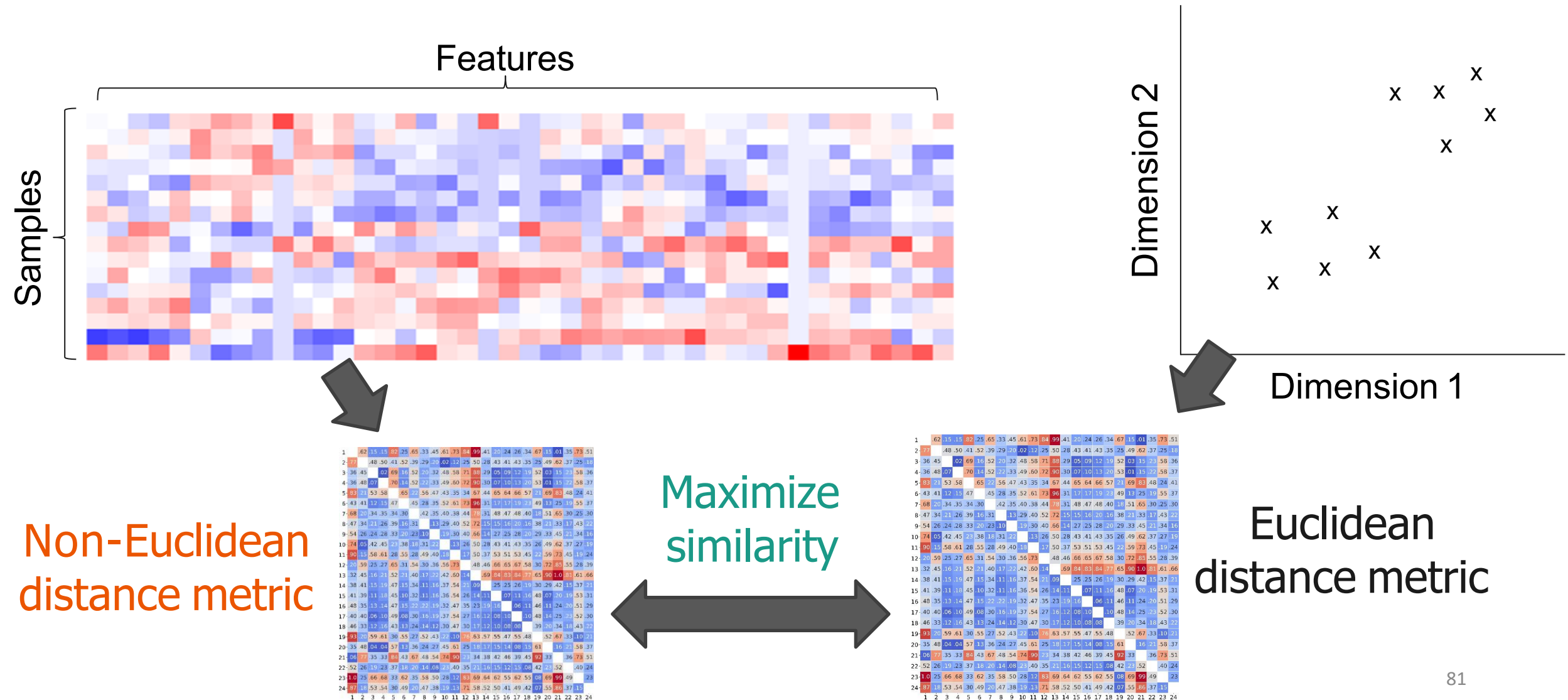


# Pairwise distance matrix

|    |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |     |
|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1  |     | .62 | .15 | .15 | .82 | .25 | .65 | .33 | .45 | .61 | .73 | .84 | .99 | .41 | .20 | .24 | .26 | .34 | .67 | .15 | .01 | .35 | .73 | .51 |
| 2  | .77 |     | .48 | .50 | .41 | .52 | .39 | .29 | .20 | .02 | .12 | .25 | .50 | .28 | .43 | .41 | .43 | .35 | .25 | .49 | .62 | .37 | .25 | .18 |
| 3  | .36 | .45 |     | .02 | .69 | .16 | .52 | .20 | .32 | .48 | .58 | .71 | .88 | .29 | .05 | .09 | .12 | .19 | .52 | .03 | .15 | .23 | .58 | .36 |
| 4  | .36 | .48 | .07 |     | .70 | .14 | .52 | .22 | .33 | .49 | .60 | .72 | .90 | .30 | .07 | .10 | .13 | .20 | .53 | .01 | .15 | .22 | .58 | .37 |
| 5  | .83 | .21 | .53 | .58 |     | .65 | .22 | .56 | .47 | .43 | .35 | .34 | .67 | .44 | .65 | .64 | .66 | .57 | .21 | .69 | .83 | .48 | .24 | .41 |
| 6  | .43 | .41 | .12 | .15 | .47 |     | .45 | .28 | .35 | .52 | .61 | .73 | .96 | .31 | .17 | .17 | .19 | .23 | .49 | .13 | .25 | .19 | .55 | .37 |
| 7  | .68 | .20 | .34 | .35 | .34 | .30 |     | .42 | .35 | .40 | .38 | .44 | .78 | .31 | .48 | .47 | .48 | .40 | .18 | .51 | .65 | .30 | .25 | .30 |
| 8  | .47 | .34 | .21 | .26 | .39 | .16 | .31 |     | .13 | .29 | .40 | .52 | .72 | .15 | .15 | .16 | .20 | .16 | .38 | .21 | .33 | .17 | .43 | .22 |
| 9  | .54 | .26 | .24 | .28 | .33 | .20 | .23 | .10 |     | .19 | .30 | .40 | .66 | .14 | .27 | .25 | .28 | .20 | .29 | .33 | .45 | .21 | .34 | .16 |
| 10 | .74 | .05 | .42 | .45 | .23 | .38 | .18 | .31 | .22 |     | .13 | .26 | .50 | .28 | .43 | .41 | .43 | .35 | .26 | .49 | .62 | .37 | .27 | .19 |
| 11 | .90 | .15 | .58 | .61 | .28 | .55 | .28 | .49 | .40 | .18 |     | .17 | .50 | .37 | .53 | .51 | .53 | .45 | .22 | .59 | .73 | .45 | .19 | .24 |
| 12 | .20 | .59 | .25 | .27 | .65 | .31 | .54 | .30 | .36 | .56 | .73 |     | .48 | .46 | .66 | .65 | .67 | .58 | .30 | .72 | .85 | .55 | .28 | .39 |
| 13 | .32 | .45 | .16 | .21 | .52 | .21 | .40 | .17 | .22 | .42 | .60 | .14 |     | .69 | .84 | .83 | .84 | .77 | .65 | .90 | 1.0 | .81 | .61 | .66 |
| 14 | .38 | .41 | .15 | .19 | .47 | .15 | .34 | .11 | .16 | .37 | .54 | .21 | .09 |     | .25 | .25 | .26 | .19 | .30 | .29 | .42 | .15 | .37 | .21 |
| 15 | .41 | .39 | .11 | .18 | .45 | .10 | .32 | .11 | .16 | .36 | .54 | .26 | .14 | .11 |     | .07 | .11 | .16 | .48 | .07 | .20 | .19 | .53 | .31 |
| 16 | .48 | .35 | .13 | .14 | .47 | .15 | .22 | .22 | .19 | .32 | .47 | .35 | .23 | .19 | .16 |     | .06 | .11 | .46 | .11 | .24 | .20 | .51 | .29 |
| 17 | .40 | .40 | .06 | .10 | .49 | .08 | .30 | .16 | .19 | .37 | .54 | .27 | .16 | .12 | .08 | .10 |     | .10 | .48 | .14 | .25 | .23 | .52 | .30 |
| 18 | .46 | .33 | .12 | .16 | .43 | .13 | .24 | .14 | .12 | .30 | .47 | .30 | .17 | .12 | .10 | .08 | .08 |     | .39 | .20 | .34 | .18 | .43 | .22 |
| 19 | .93 | .20 | .59 | .61 | .30 | .55 | .27 | .52 | .43 | .22 | .10 | .76 | .63 | .57 | .55 | .47 | .55 | .48 |     | .52 | .67 | .33 | .10 | .21 |
| 20 | .35 | .48 | .04 | .04 | .57 | .13 | .36 | .24 | .27 | .45 | .61 | .25 | .18 | .17 | .15 | .14 | .08 | .15 | .61 |     | .16 | .21 | .58 | .37 |
| 21 | .06 | .77 | .35 | .33 | .84 | .43 | .67 | .48 | .54 | .74 | .90 | .23 | .34 | .38 | .42 | .46 | .39 | .45 | .92 | .33 |     | .36 | .73 | .51 |
| 22 | .52 | .26 | .19 | .23 | .37 | .18 | .20 | .14 | .08 | .23 | .40 | .35 | .21 | .16 | .15 | .12 | .15 | .08 | .42 | .23 | .52 |     | .40 | .24 |
| 23 | 1.0 | .25 | .66 | .68 | .33 | .62 | .35 | .58 | .50 | .28 | .12 | .83 | .69 | .64 | .62 | .55 | .62 | .55 | .08 | .69 | .99 | .49 |     | .23 |
| 24 | .87 | .18 | .53 | .54 | .30 | .49 | .20 | .47 | .38 | .19 | .13 | .71 | .58 | .52 | .50 | .41 | .49 | .42 | .07 | .55 | .86 | .37 | .15 |     |
|    | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12  | 13  | 14  | 15  | 16  | 17  | 18  | 19  | 20  | 21  | 22  | 23  | 24  |

- $D(i, j)$  = distance between sample  $i$  and sample  $j$
- $D(i, i) = 0$
- $D(i, j) = D(j, i)$
- User-defined distance metric

# Principal Coordinate Analysis (PCoA)



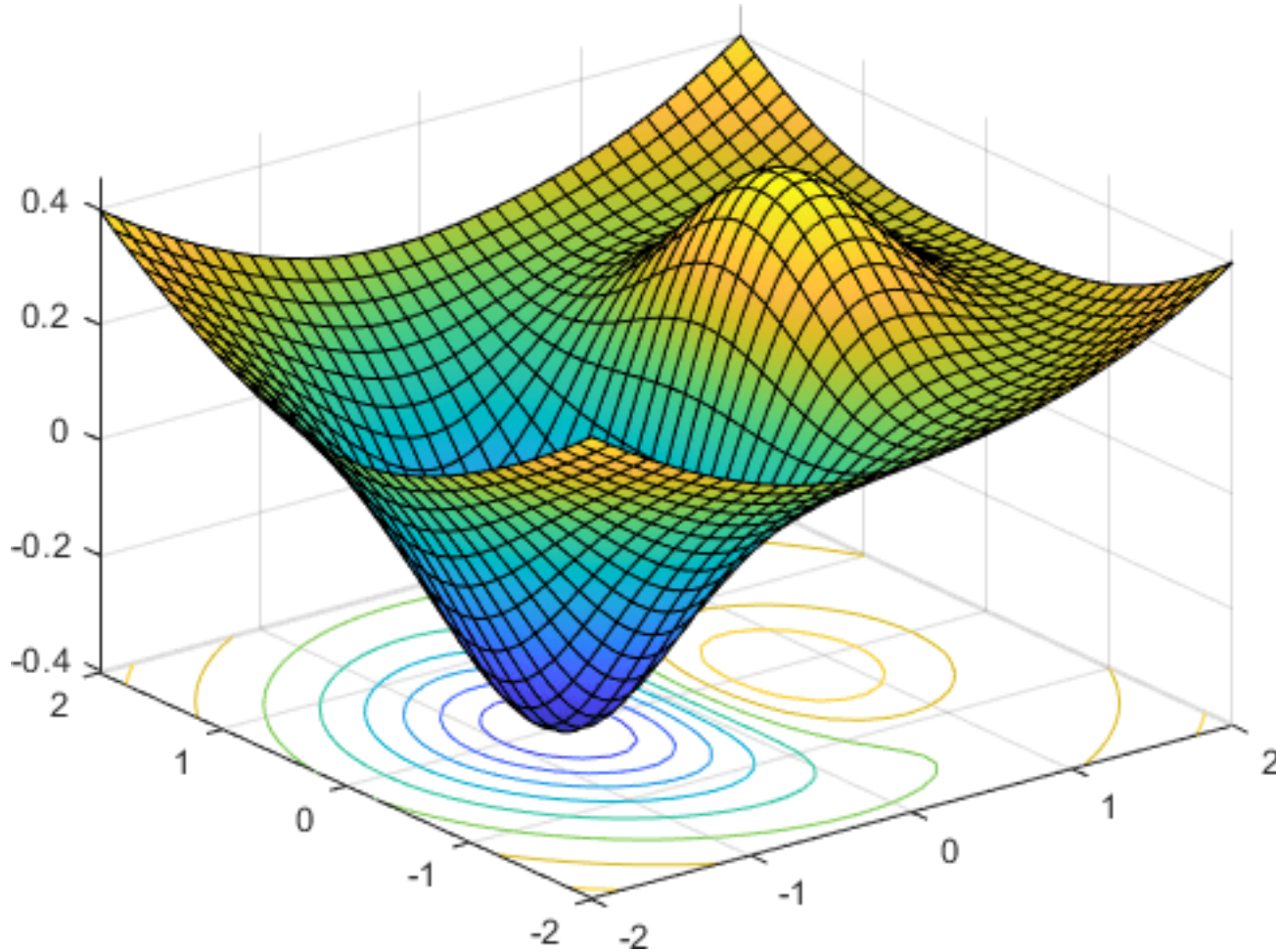


# PCoA algorithm sketch

- Calculate pairwise distance matrix  $D$  for the original data
- For a 2D projection of the data: sample  $i$  projected onto  $(x_i, y_i)$ 
  - Calculate pairwise distance matrix  $D'$  for the projection
  - Each element of  $D'$  is a function of  $(x_i, y_i)$ 's
- Calculate a similarity score between  $D$  and  $D'$ 
  - Such as Person's correlation
  - This similarity score is a function of  $(x_i, y_i)$ 's
- Find  $(x_i, y_i)$ 's that maximize this score!

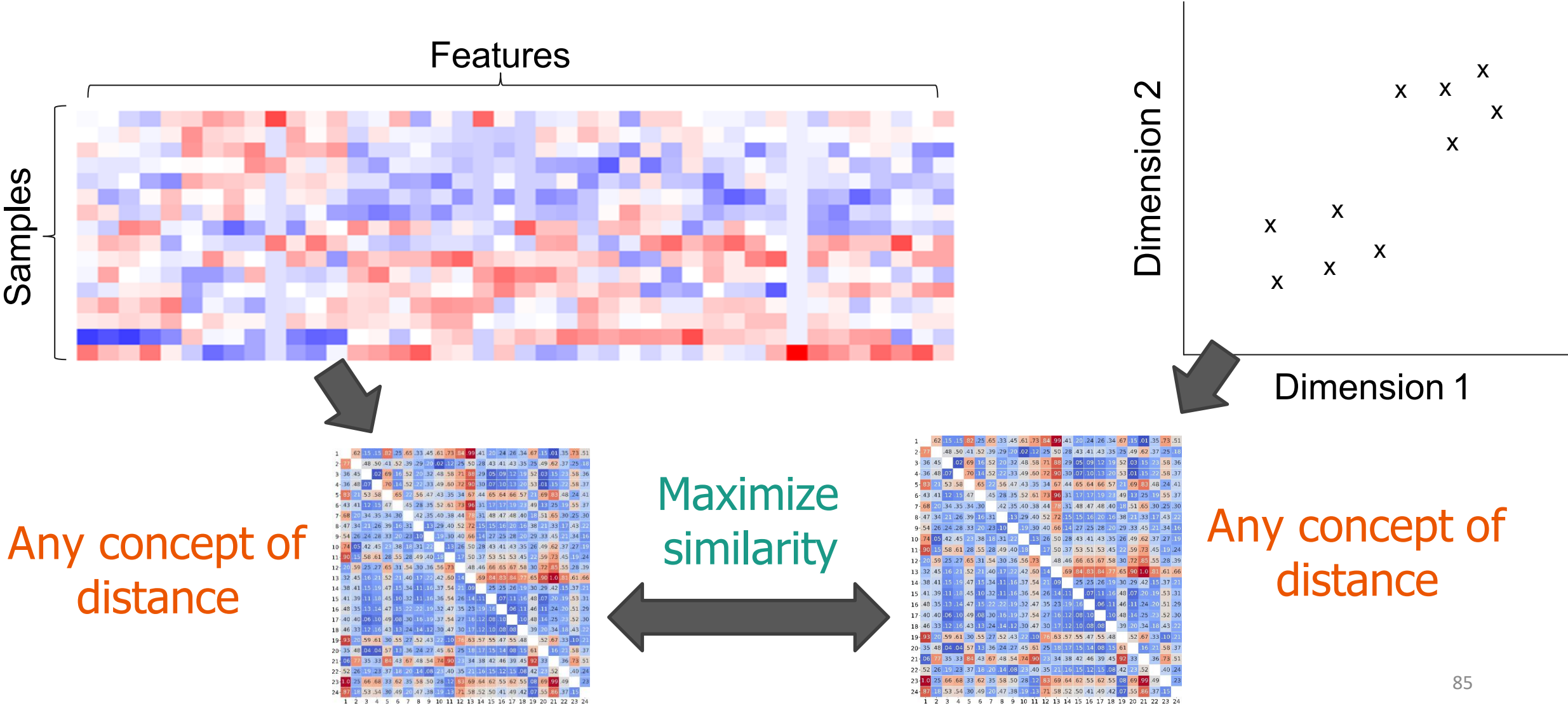


# How to optimize a function?

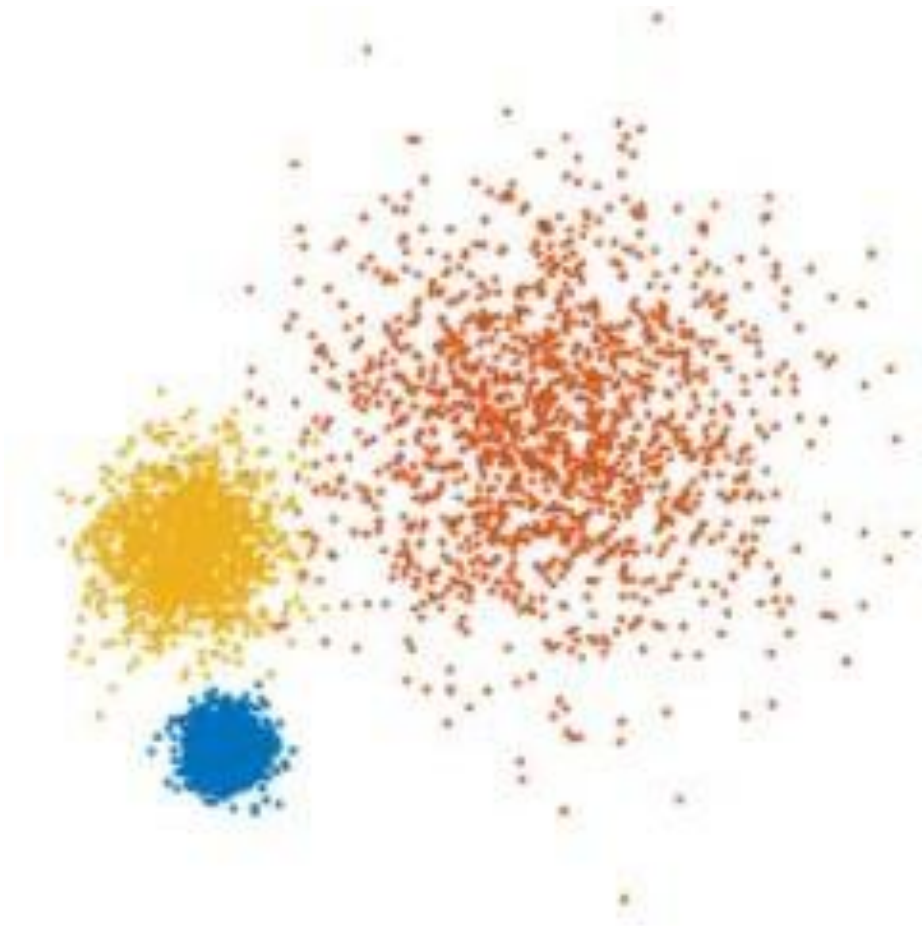


- Find  $(x_1, x_2, \dots, x_n)$  that minimize  $f(x_1, x_2, \dots, x_n)$
- At minimum, the slope is zero in all directions
- Take derivative of each variable and set to zero
  - $\frac{\delta f}{\delta x_1} = 0$
  - $\frac{\delta f}{\delta x_2} = 0$
  - $n$  equations with  $n$  variables

# Generalized MDS



# Limitation of PCA and MDS



- A single definition of distance metric is used throughout the data space
- What if some data groups are noisier than the others?
  - Difference in **data density**

# MDS in Python

## `sklearn.manifold.MDS`

```
class sklearn.manifold.MDS(n_components=2, *, metric=True, n_init=4, max_iter=300, verbose=0, eps=0.001, n_jobs=None, random_state=None, dissimilarity='euclidean')
```

[\[source\]](#)

- `n_components` = number of dimensions to embed onto
- Set `metric` = True if using a known distance metric
- Set `metric` = False if inputting a custom dissimilarity that might be a metric
- `dissimilarity` indicates how the sample-sample distance/dissimilarity should be calculated
  - Use `precomputed` to input your own custom dissimilarity

# MDS usage

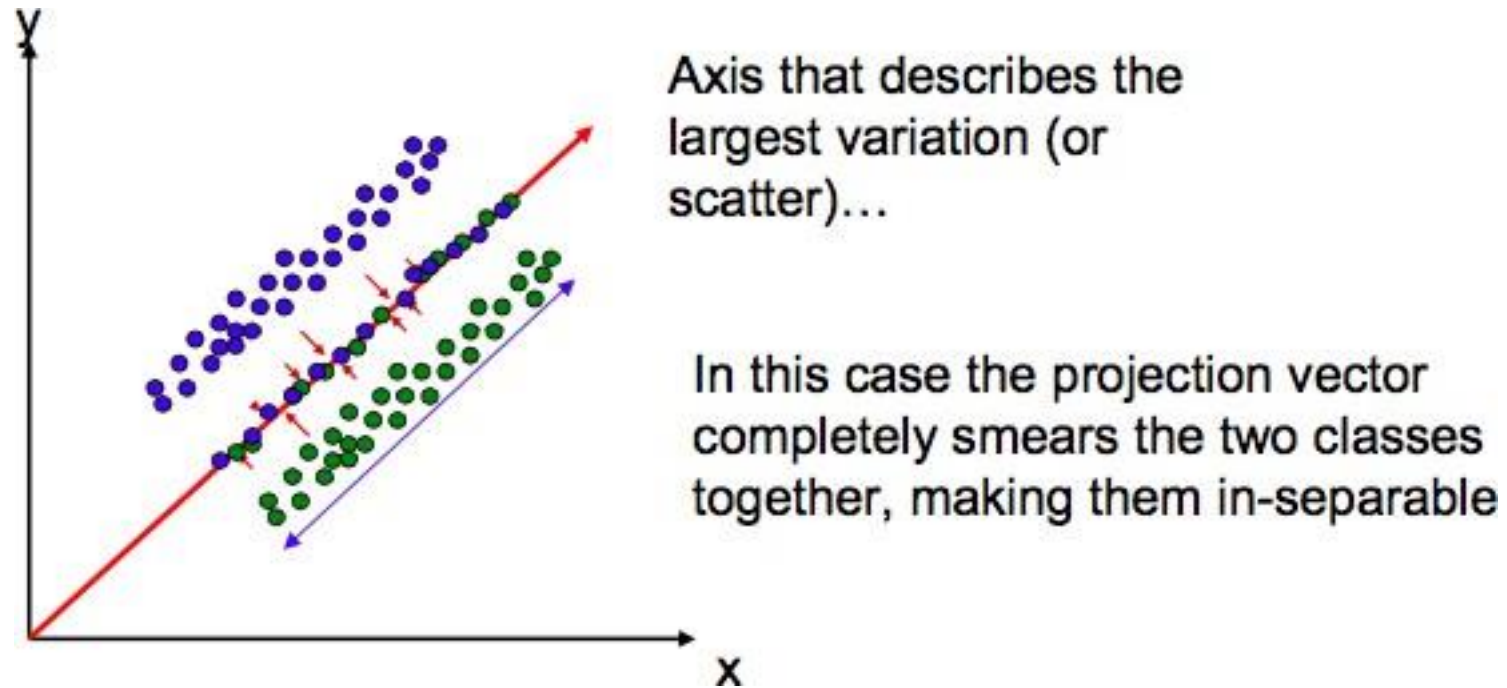
```
from sklearn.manifold import MDS
from scipy.spatial.distance import pdist, squareform

## calculate pairwise distance matrix between all samples
corr_mat = squareform(pdist(data, metric = 'correlation'))

## embed with MDS's 'precomputed' option
mds = MDS(n_components = 2, dissimilarity = 'precomputed', random_state =
25) embed = mds.fit_transform(corr_mat)
```

# PCA for classification

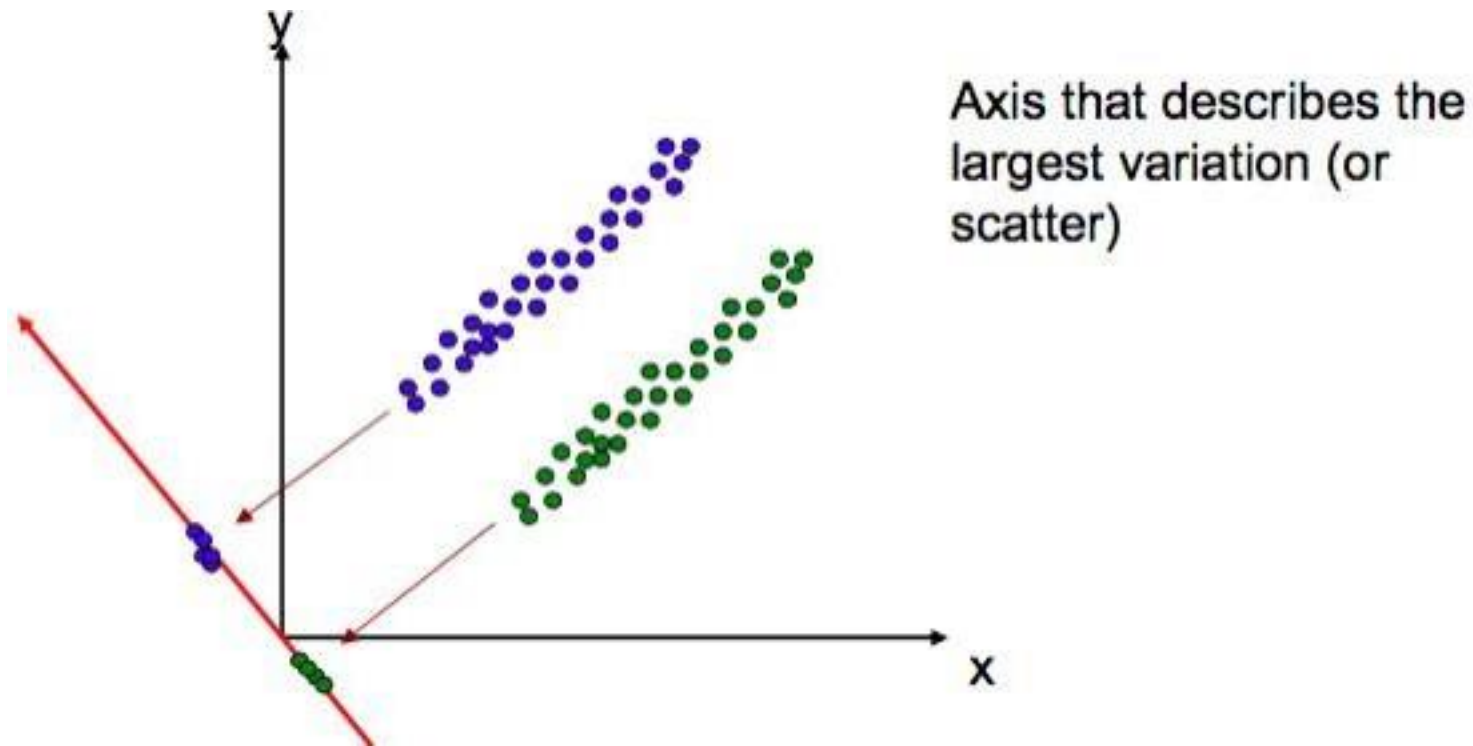
- PCA does not care about the class labels





# What is Linear Discriminant Analysis (LDA)

- Find the projections that separate the classes.
- Assumes unimodal Gaussian model for each class
  - Maximize the distance between the means and minimize the variance of each class -> best classification performance



# Simple 2 class case

- We want to maximize the distance between the projected means:

e.g. maximize  $|(\tilde{\mu}_1 - \tilde{\mu}_2)|^2$

Where  $\tilde{\mu}_1$  is the projected mean  $\mu_1$  of class onto LDA direction vector  $\mathbf{w}$ , i.e.

$$\tilde{\mu}_1 = \mathbf{w}^T \mu_1$$

and for class 2:  $\tilde{\mu}_2 = \mathbf{w}^T \mu_2$  thus

$$\begin{aligned} |(\tilde{\mu}_1 - \tilde{\mu}_2)|^2 &= |(\mathbf{w}^T \mu_1 - \mathbf{w}^T \mu_2)|^2 \\ &= \mathbf{w}^T (\mu_1 - \mu_2) (\mu_1 - \mu_2)^T \mathbf{w} \\ &= \mathbf{w}^T \mathbf{S}_B \mathbf{w} \end{aligned}$$



Between class scatter matrix  $S_B$

$$\begin{aligned}(\tilde{\mu}_1 - \tilde{\mu}_2)^2 &= (\mathbf{w}^T \boldsymbol{\mu}_1 - \mathbf{w}^T \boldsymbol{\mu}_2)^2 \\&= \mathbf{w}^T (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T \mathbf{w} \\&= \mathbf{w}^T \mathbf{S}_B \mathbf{w}\end{aligned}$$

We want to maximize  $\mathbf{w}^T \mathbf{S}_B \mathbf{w}$  where  $S_B$  is the between class scatter matrix defined as:

$$\mathbf{S}_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$$

# We also want to minimize within class scatter

- The variance or scatter of each class. We also want to minimize them.

$$\tilde{s}_1^2 = \sum_{i=1}^{N_1} (\tilde{x}_i - \tilde{\mu}_1)^2$$

Minimize the total scatter  $\tilde{s}_1^2 + \tilde{s}_2^2$

# Within class scatter

- Lets expand on scatter  $s_1, s_2$ .

$$\begin{aligned}\tilde{s}_1^2 &= \sum_{i=1}^{N_1} (\tilde{x}_i - \tilde{\mu}_1)^2 \\ &= \sum_{i=1}^{N_1} (\mathbf{w}^T \mathbf{x}_i - \mathbf{w}^T \boldsymbol{\mu}_1)^2 \\ &= \sum_{i=1}^{N_1} \mathbf{w}^T (\mathbf{x}_i - \boldsymbol{\mu}_1)(\mathbf{x}_i - \boldsymbol{\mu}_1)^T \mathbf{w} \\ &= \mathbf{w}^T \mathbf{S}_1 \mathbf{w}\end{aligned}$$

# Total within class scatter

- We want to minimize

$$\tilde{S}_1^2 + \tilde{S}_2^2$$

- This is the same as  $\mathbf{w}^T \mathbf{S}_w \mathbf{w}$

$$\mathbf{S}_w = \sum_{i=1}^C \sum_{j=1}^{N_i} (\mathbf{x}_j - \boldsymbol{\mu}_i)(\mathbf{x}_j - \boldsymbol{\mu}_i)^T$$

C number of classes,  $N_i$  number of images from class  $i$

# Fisher Linear Discriminant Criterion

- We want to maximize between class scatter
- We want to minimize within class scatter
- We have an objective function as a ratio so we can achieve both!

$$J(\mathbf{w}) = \frac{|(\tilde{\mu}_1 - \tilde{\mu}_2)|^2}{\tilde{s}_1^2 + \tilde{s}_2^2}$$
$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

# LDA solution

- If you do calculus

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}$$

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}$$

If  $\mathbf{S}_W$  is non-singular and invertible.

- Generalized eigenvalue problem. The number of solutions is  $\min(\text{rank} \mathbf{S}_B, \text{rank} \mathbf{S}_W) = C-1$  or  $N-C$
- For 2 class this simplifies to
  - Note this is only one projection direction

$$\mathbf{w} = \mathbf{S}_W^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$

# LDA+PCA

- Original features dimension  $L$
- $S_B$   $S_W$  are  $L \times L$  matrixes
- $\text{rank} S_B = C-1$ ,  $\text{rank} S_W = N-C$
- First do PCA to reduce dimension
- Then do LDA to maximize classification ability
- How many dimensions to PCA?
  - Do PCA to keep  $N-C$  eigenvectors  $\rightarrow$  Makes  $S_W$  full rank and invertible
  - Then, do LDA and compute  $C-1$  projections in this  $N-C$  subspace
- PCA+LDA = Fisher projection

# Random projection

- Original  $d$ -dimensional data is project to  $k$ -dimensional subspace
- Using a random  $k \times d$  matrix  $R$  with unit norm columns
  - Johnson-Lindenstrauss lemma: If points in a vector space are projected onto a randomly selected subspace of suitably high dimension, then the distances between the points are approximately preserved
- Elements of  $R$  are usually selected from Gaussians.
  - Generally any zero mean unit variance distribution would satisfy Johnson-Lindenstrauss lemma.



# Random projection notes

- R is not generally orthogonal.
  - But in a substantially large subspace, random vectors might be close to orthogonal.
- Looks weird but works...
- Robust to outliers in the data, because it uses no data
- Faster to train than PCA if data has high dimension
  - $O(k^2 \times n + k^3)$  (covariance computation + eigen decomposition)  
where k- features, n – samples
- See <https://medium.com/data-science-in-your-pocket/random-projection-for-dimension-reduction-27d2ec7d40cd>  
for usage guides

# Visualization

## Methods covered

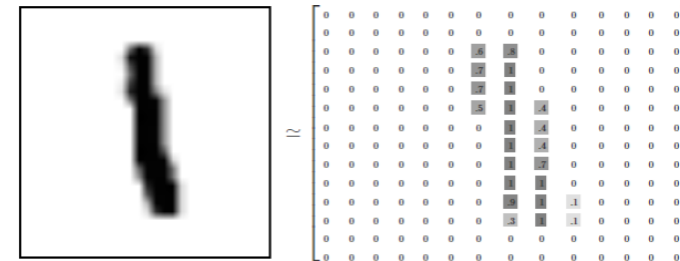
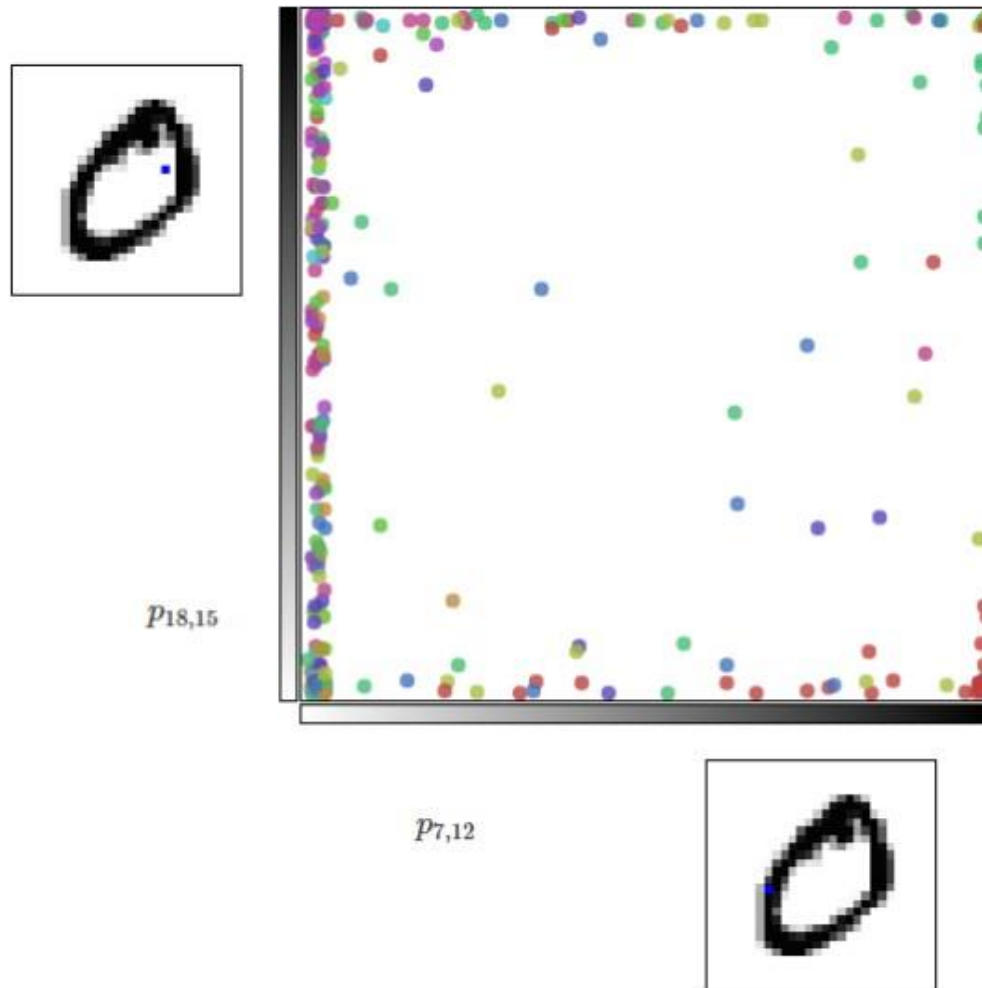
1. Reducing reconstruction error: PCA
2. Keeping direction of maximum separability: LDA
3. Preserving distance (globally): RP

These are usually useful for downstream machine learning methods. (Classification/Regression/Clustering)

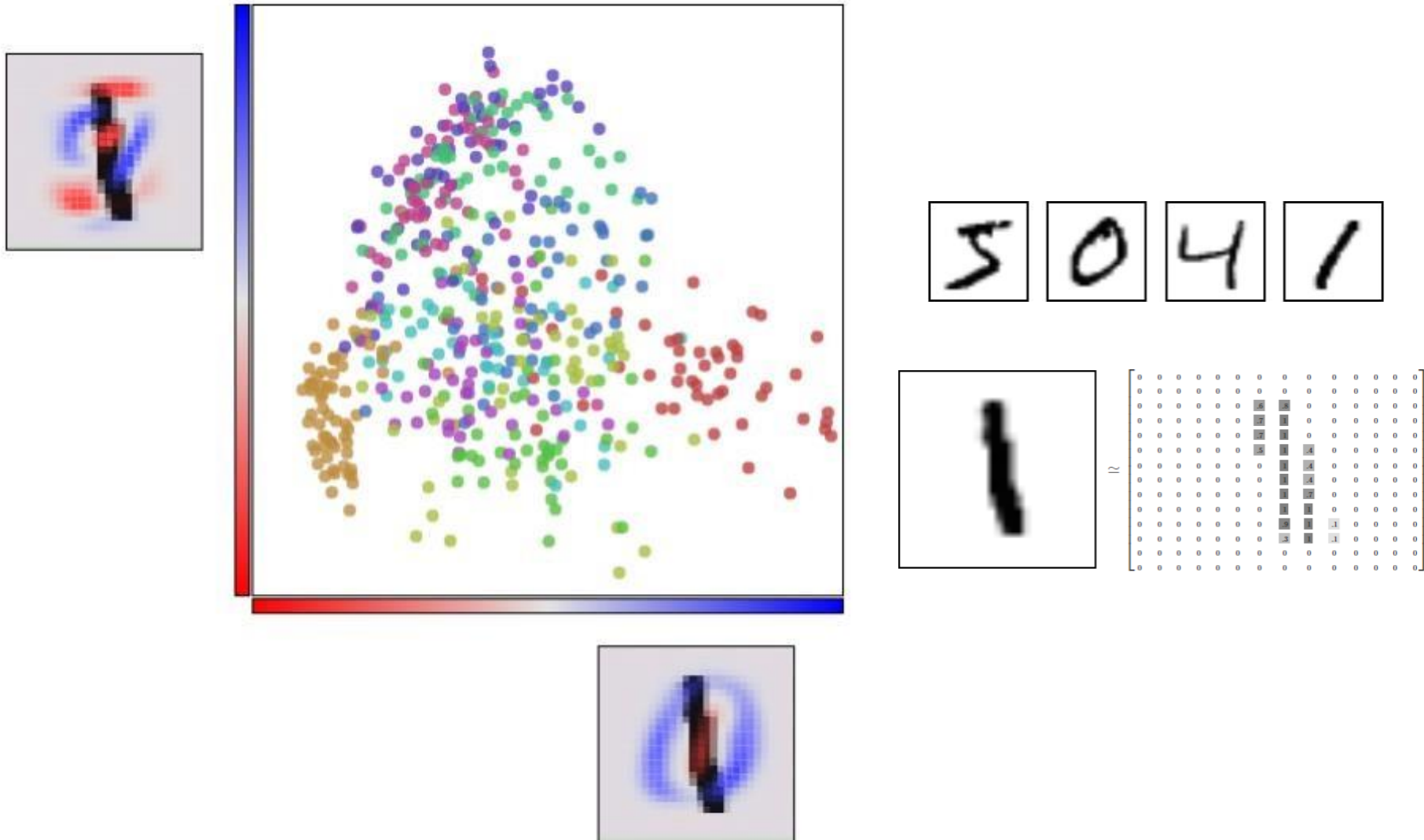
But what if we, as humans, want to get a sense of our data?

1. Interpretability (in some sense): ????
  - Check if our dimensionality reduction is good
  - Analyze clusters from k-mean, GMM, etc.

# Visualizing MNIST



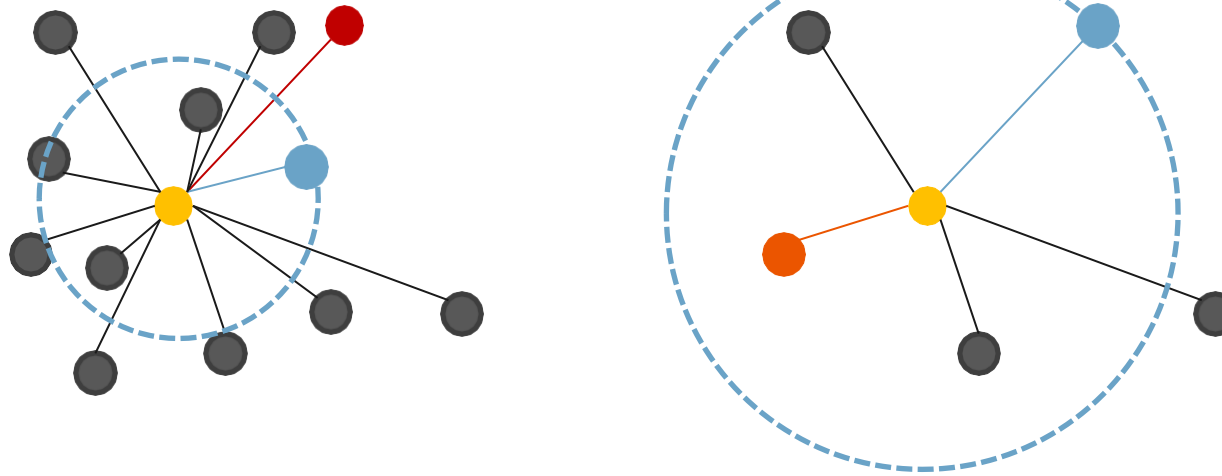
# PCA with MNIST



# t-distributed Stochastic Neighbor Embedding (t-SNE)

Preserves neighbor (preserves local distance).

- Things close together should be close together in the projected space
- Prefer using few projected dimensions (2-3)



# Defining neighbors

Define  $P_{j|i}$  probability that  $i$  would pick  $j$  as its neighbor

Assume  $i$  picks proportional to Gaussian centered at  $i$

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2) / 2\sigma_i^2}{\sum_{k \neq i} \exp(-||x_i - x_k||^2) / 2\sigma_i^2}$$

$P_{i|i} = 0$  since we don't want to have it pick itself

The variance is fixed to some value.

# Defining neighbors

Define  $p_{j|i}$  probability that  $i$  would pick  $j$  as its neighbor

Assume  $i$  picks proportional to Gaussian centered at  $i$

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2) / 2\sigma_i^2}{\sum_{k \neq i} \exp(-||x_i - x_k||^2) / 2\sigma_i^2}$$

When projected to set of points  $\{y_i\}$ , define  $q_{j|i}$  the probability that  $i$  would pick  $j$  in embedding/latent space

$$q_{j|i} = \frac{\exp(-||y_i - y_j||^2)}{\sum_{k \neq i} \exp(-||y_i - y_k||^2)}$$

We set the variance in the  $y$  space to be  $1/\text{sqrt}(2)$

# Defining neighbors

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2) / 2\sigma_i^2}{\sum_{k \neq i} \exp(-||x_i - x_k||^2) / 2\sigma_i^2}$$

$$q_{j|i} = \frac{\exp(-||y_i - y_j||^2)}{\sum_{k \neq i} \exp(-||y_i - y_k||^2)}$$

We expect p and q to be the same -> small distance

How to measure distance between probability functions?

Kullback-Leibler (KL) divergence



# KL divergence

Distance between two distributions

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} = - \sum_i P(i) \log \frac{Q(i)}{P(i)}$$

Note  $D_{KL}(P||Q) \neq D_{KL}(Q||P)$  (Not a real distance)

Always positive. Equals 0 iff  $Q = P$  at every point.

$$P(\text{head}) = 0.5 \quad P(\text{tail}) = 0.5$$

$$Q(\text{head}) = 0.7 \quad Q(\text{tail}) = 0.3$$

$$D_{KL}(P||Q) = 0.5 \cdot \ln 0.5/0.7 + 0.5 \cdot \ln 0.5/0.3 = 0.087$$

$$D_{KL}(Q||P) = 0.7 \cdot \ln 0.7/0.5 + 0.3 \cdot \ln 0.3/0.5 = 0.082$$

# Loss function

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2) / 2\sigma_i^2}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2) / 2\sigma_i^2} \quad q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}$$

We expect p and q to be the same -> small distance

Loss function

All points i      KL computes over j

$$\sum_i D_{KL}(p_i || q_i)$$
$$D_{KL}(P||Q) = \sum_j \boxed{P(j)} \log \frac{P(j)}{Q(j)}$$

Note P can be considered as the weight for the distance

Where p is large but q is small -> large penalty

p is small but q is large -> small penalty

**D(p||q) focuses on local structure in p**

What are we minimizing wrt?

How to minimize loss?

# From SNE to t-SNE

Symmetric density  
t-distributed

# Symmetric with joint probability

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

$$p_{j|i} \neq p_{i|j}$$

A point  $i$  far away from everything will have small  $p_{j|i}$

- Location of points in  $q$  no longer matter

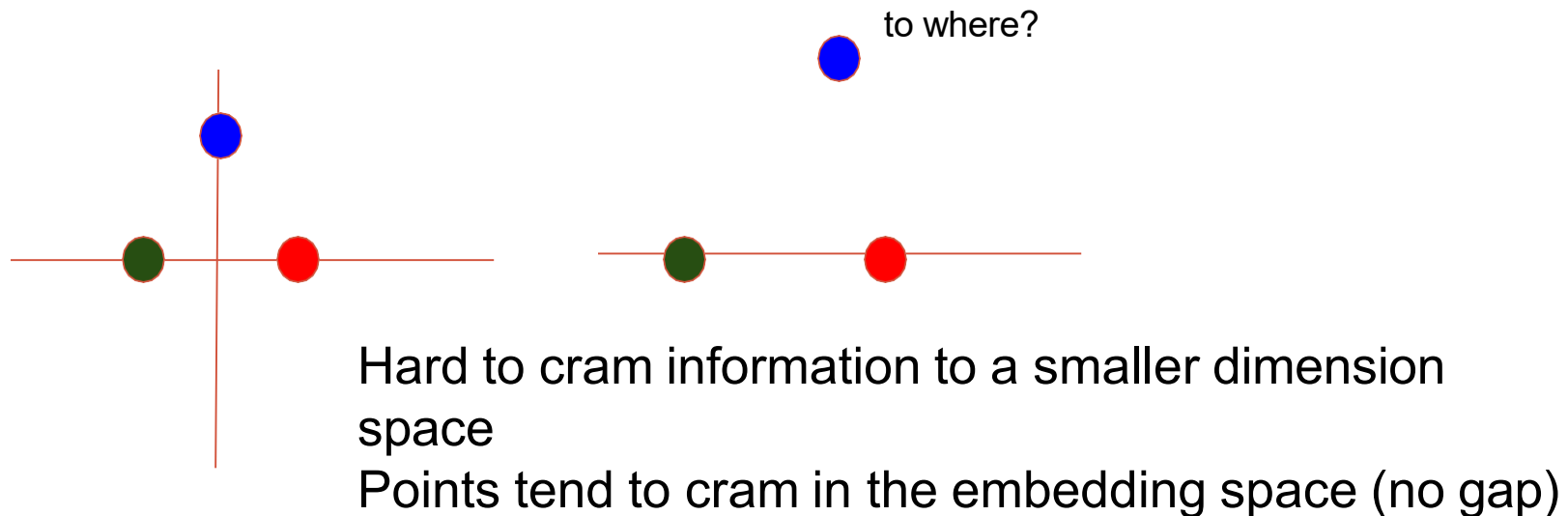
Create “joint probability”  $p_{ij} = p_{ji} = (p_{i|j} + p_{j|i})/2$

- Each data point will contribute to the loss

Use instead of conditional probability in KL divergence

# t-distributed

- “Crowding problem”
- In  $N$  dimension, you can have  $N+1$  points at equal distance. But you cannot model this in smaller dimension

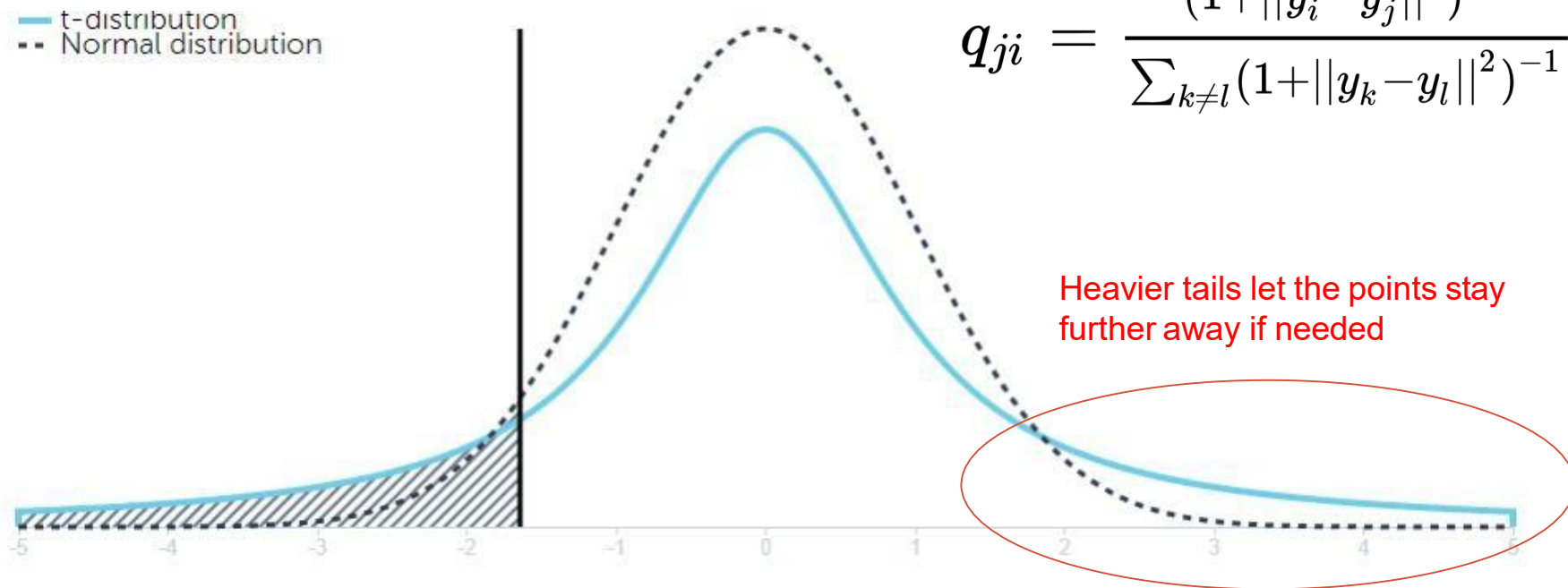


# t-distributed

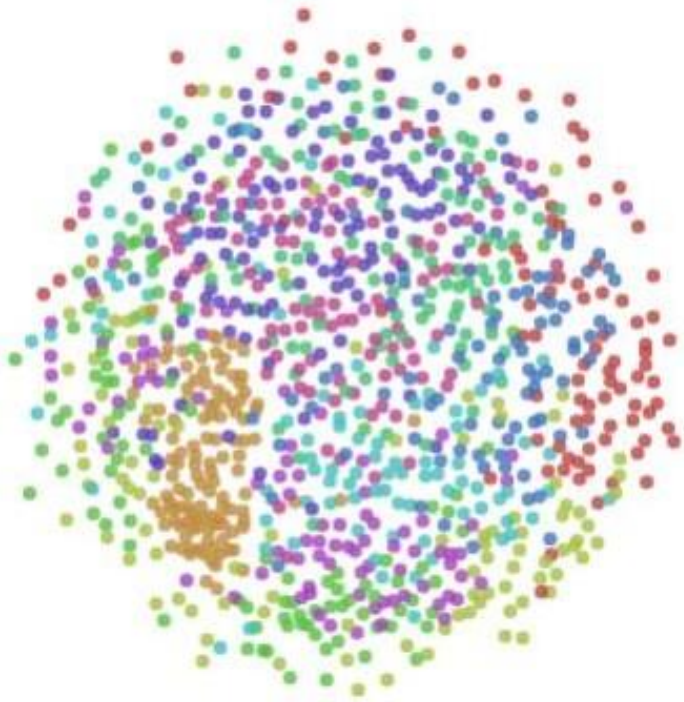
- Instead of Gaussian for  $q$  we use student's t distribution

$$q_{ji} \propto (1 + ||\mathbf{y}_i - \mathbf{y}_j||^2)^{-1}$$

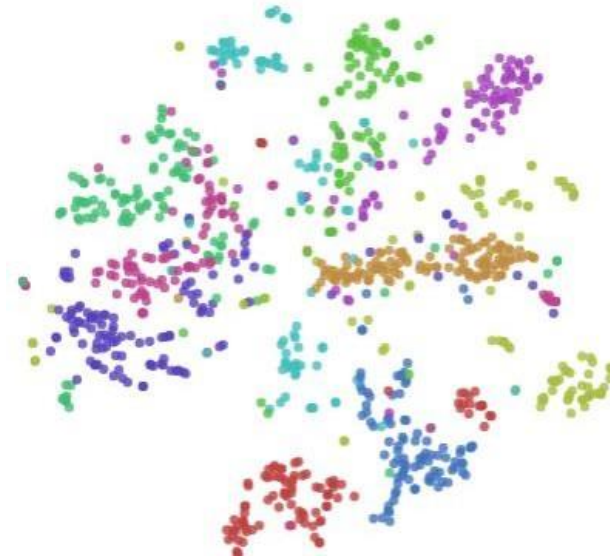
$$q_{ji} = \frac{(1 + ||\mathbf{y}_i - \mathbf{y}_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||\mathbf{y}_k - \mathbf{y}_l||^2)^{-1}}$$



# Crowding and t-SNE



Sammon's mapping  
(crowding problem)



t-SNE

# Variance

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2) / 2\sigma_i^2}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2) / 2\sigma_i^2}$$

How to set the variance of our original space?

A single variance for all points is not ideal.

- Want small variance for dense parts
- Want big variance for sparse parts

Set variance by amount of neighbors you want!

How to quantify amount of neighbors?



# Perplexity

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2) / 2\sigma_i^2}{\sum_{k \neq i} \exp(-||x_i - x_k||^2) / 2\sigma_i^2}$$

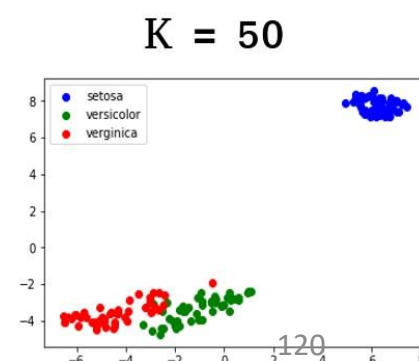
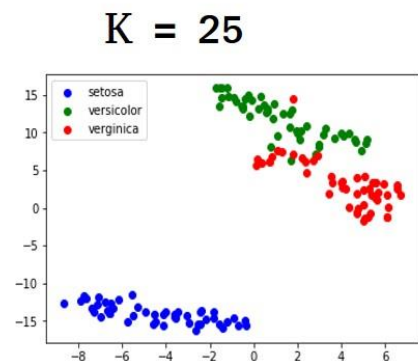
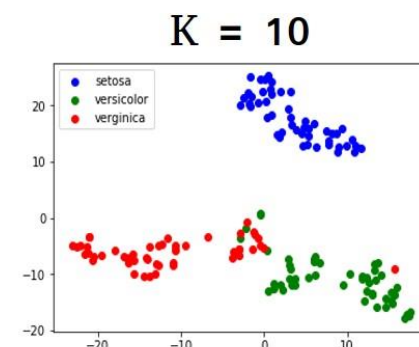
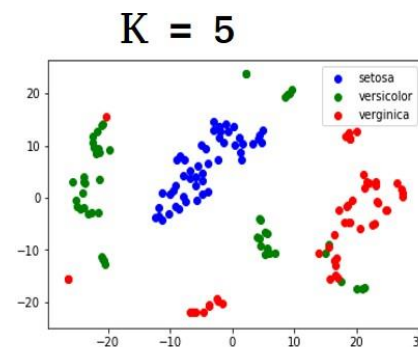
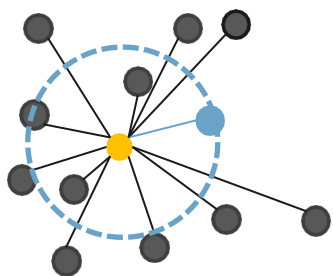
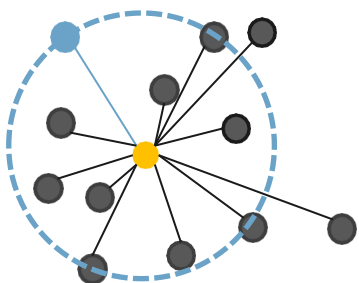
$$\text{Perp}(P_i) = 2^{H(P_i)}$$

$$H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i} \quad \text{Entropy}$$

Perplexity of  $P_i$  represents effective amount of neighbors for the point  $i$

Set  $\text{Perp}(P_i)$  then t-SNE algorithm searches for the corresponding variance

Typical values for perplexity 5 to 50



# t-SNE summary

Goal: preserves local neighbors

Gradient-based -> need multiple runs to see the best

Two parameters: #iteration, perplexity

**Does not learn a projection** (unlike PCA, LDA, RP)

- If you have a new sample, you have to re-run the whole thing
- Used for visualization only

# t-SNE in Python

## `sklearn.manifold.TSNE`

```
class sklearn.manifold.TSNE(n_components=2, *, perplexity=30.0, early_exaggeration=12.0, learning_rate='warn', n_iter=1000, n_iter_without_progress=300, min_grad_norm=1e-07, metric='euclidean', metric_params=None, init='warn', verbose=0, random_state=None, method='barnes_hut', angle=0.5, n_jobs=None, square_distances='deprecated')
```

[source]

- **n\_components** = number of dimensions to embed onto
- **perplexity** ~ number of nearby data points to use when calculating density
  - Recommend trying several values from 5 to 50
- **metric** defines the distance/dissimilarity
  - Use **precomputed** to input your own custom dissimilarity
  - For list of available distance, see [scipy.spatial.distance.pdist](#)

# t-SNE usage

```
from sklearn.manifold import TSNE
```

```
tsne = TSNE(n_components = 2, perplexity = 25, metric = 'euclidean', random_state = 25)  
embed = tsne.fit_transform(data)
```

```
## custom dissimilarity
```

```
from scipy.spatial.distance import pdist, squareform
```

```
## calculate pairwise distance matrix between all samples
```

```
corr_mat = squareform(pdist(data, metric =  
'correlation'))
```

```
## 'precomputed' option
```

```
tsne = TSNE(n_components = 2, perplexity = 25, metric = 'precomputed', random_state = 25)  
embed = tsne.fit_transform(corr_mat)
```

# UMAP: Uniform Manifold Approximation and Projection

t-SNE loses structure of important feature dimension or structure (global vs local structure)

Draw upon “inspiration” from Manifold learning theory

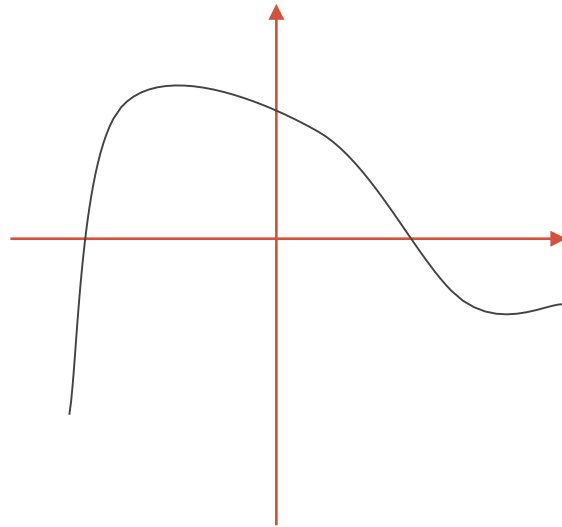
# UMAP big picture

1. Define a new kind of probability for being neighbor
  - a. Based on distance to neighbors
2. Do SGD just like t-SNE

# Manifold

Manifold: a non-linear space but locally linear

Data can be represented in 2-d but lies in 1-d manifold



UMAP defines a manifold based on the neighbors of points

# UMAP's distance function

Given fixed  $k$  (number of neighbors)

Let  $X_1, \dots, X_k$  be the  $k$ -nearest neighbors of  $X$

The distance between  $X_i$  and  $X_j$  is

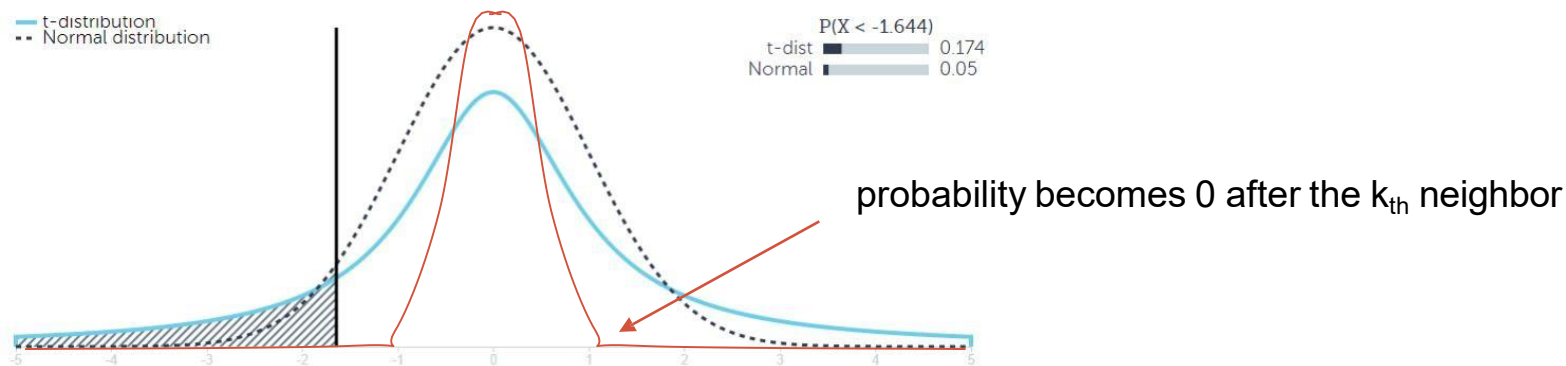
$$w_i(X_i, X_j) = \exp(-(d(X_i, X_j) - \rho_i)/\sigma_i)$$

Think of this  $w$  as giving  
"probability" that two points are  
connected

L2 distance in  
original space

distance to nearest  
neighbor so it starts  
from 0

diameter of the  
neighbors so it  
maxes at 1





# UMAP's distance function

Given fixed  $k$  (number of neighbors)

Let  $X_1, \dots, X_k$  be the  $k$ -nearest neighbors of  $X$

The distance between  $X_i$  and  $X_j$  is

$$w_i(X_i, X_j) = \exp(-(d(X_i, X_j) - \rho_i)/\sigma_i)$$

diameter of the  
neighbors so it  
maxes at 1



L2 distance in  
original space

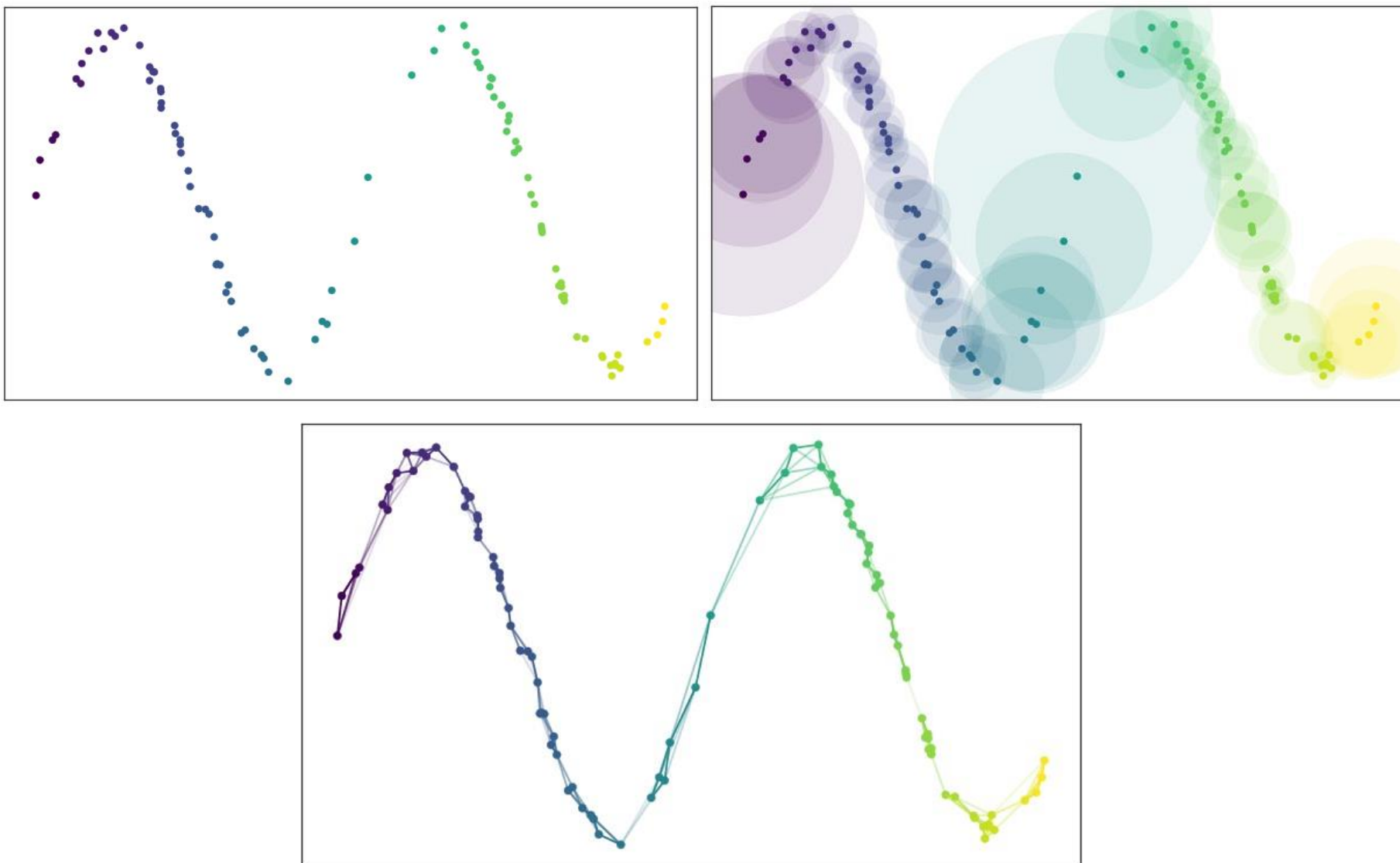


distance to nearest  
neighbor so it starts  
from 0



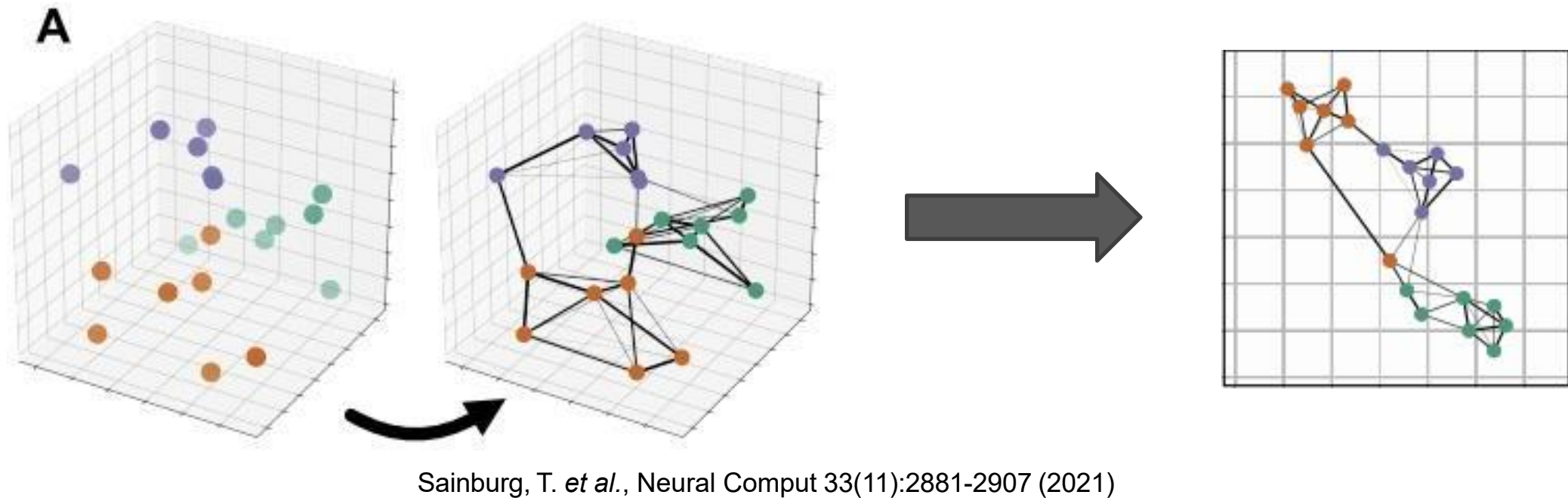
Make the distance symmetric by

$$w(X_i, X_j) = w_i(X_i, X_j) + w_j(X_j, X_i) - w_i(X_i, X_j)w_j(X_j, X_i)$$



[https://umap-learn.readthedocs.io/en/latest/how\\_umap\\_works.html](https://umap-learn.readthedocs.io/en/latest/how_umap_works.html)

# Projecting network representation



- Preserve scores on edges: probability of being neighbors

# UMAP optimization

From the weights minimize the cross-entropy between original and projected points (in terms of  $w$ )

$$\sum_{e \in E} w_h(e) \log\left(\frac{w_h(e)}{w_l(e)}\right) + (1 - w_h(e)) \log\left(\frac{1 - w_h(e)}{1 - w_l(e)}\right)$$

Distance in original space

All neighbors

Distance in new space

Similar to KL metric in t-SNE

Loss function

$$\sum_i D_{KL}(p_i || q_i)$$

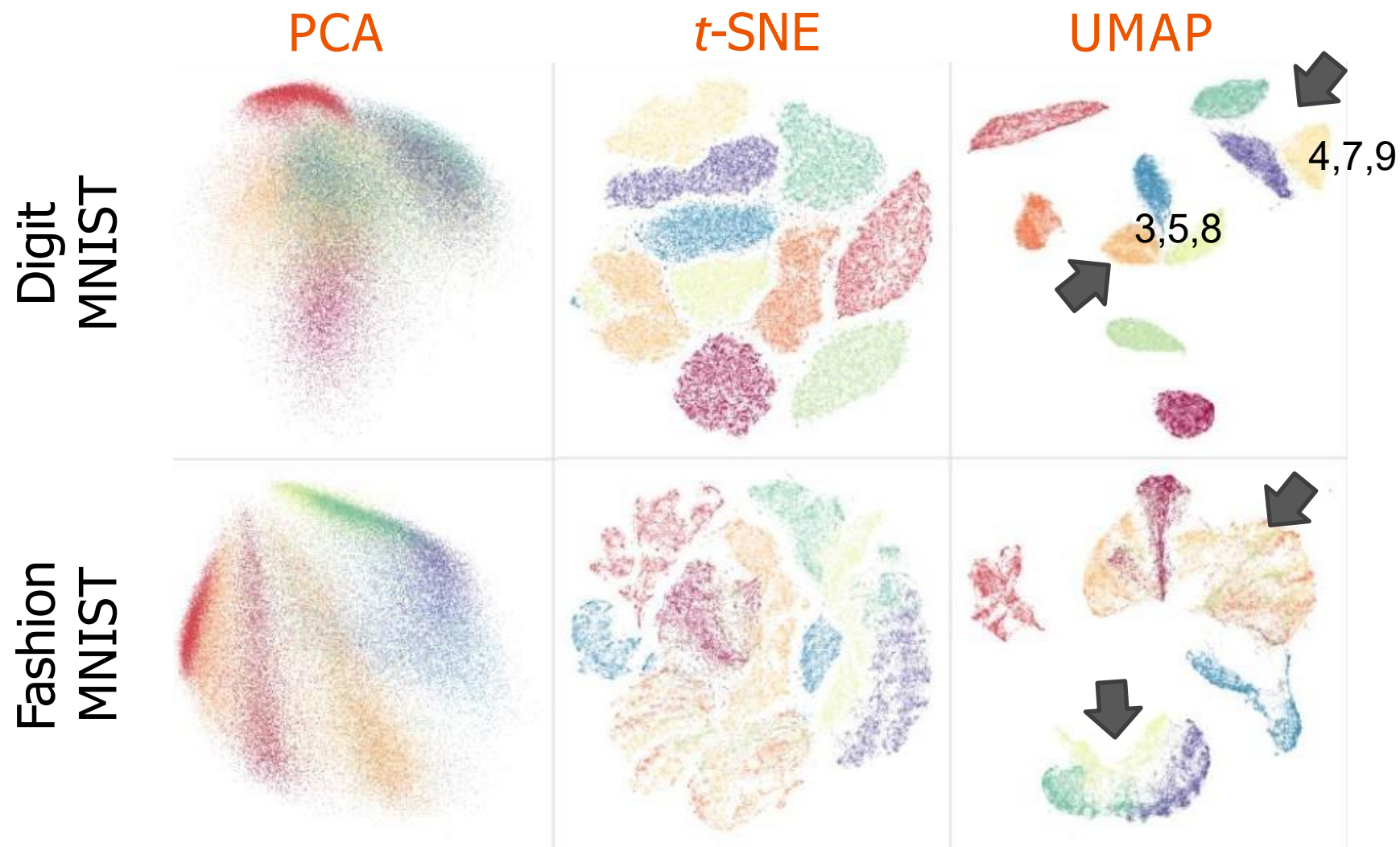
All points  $i$

KL computes over  $j$

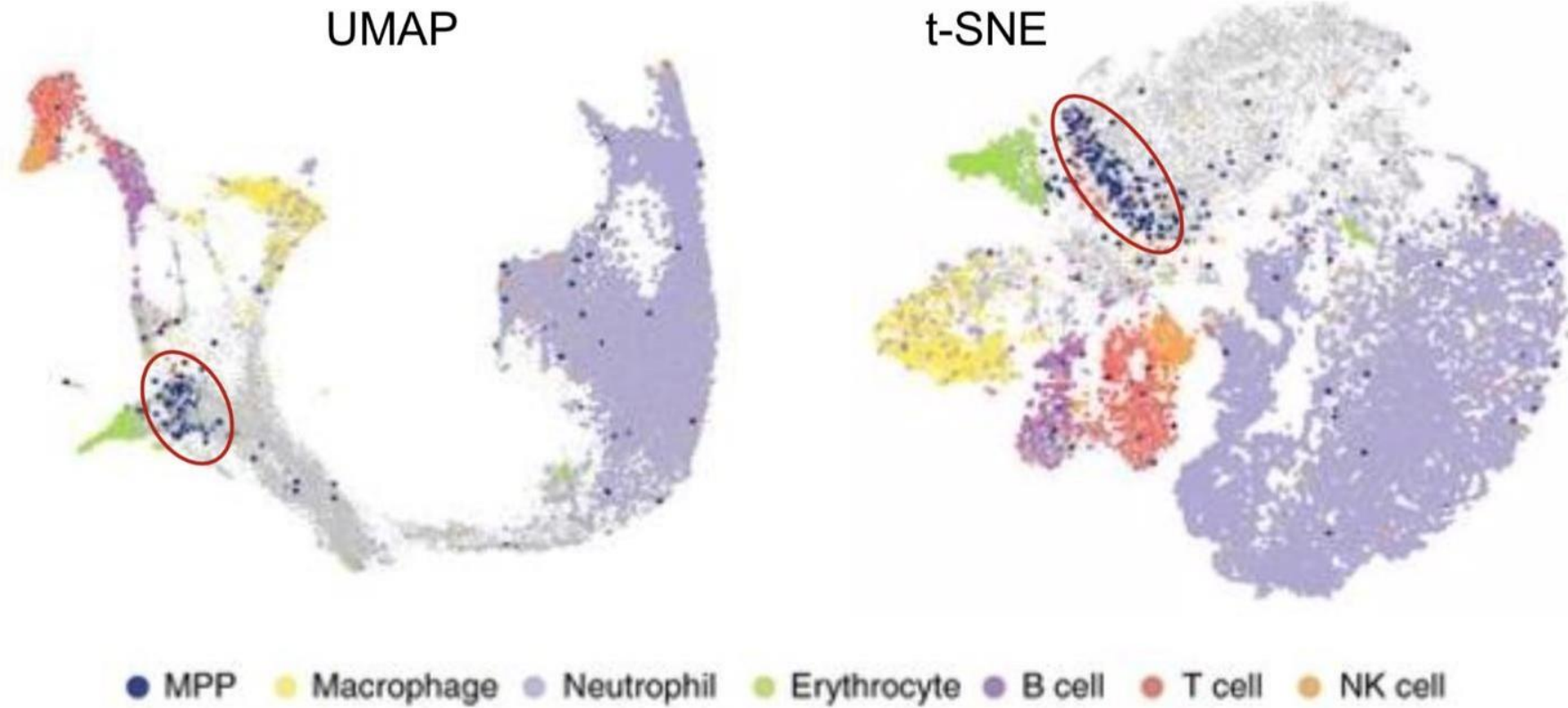
$$D_{KL}(P||Q) = \sum_j P(j) \log \frac{P(j)}{Q(j)}$$

Use gradient descent to optimize

# Power of UMAP

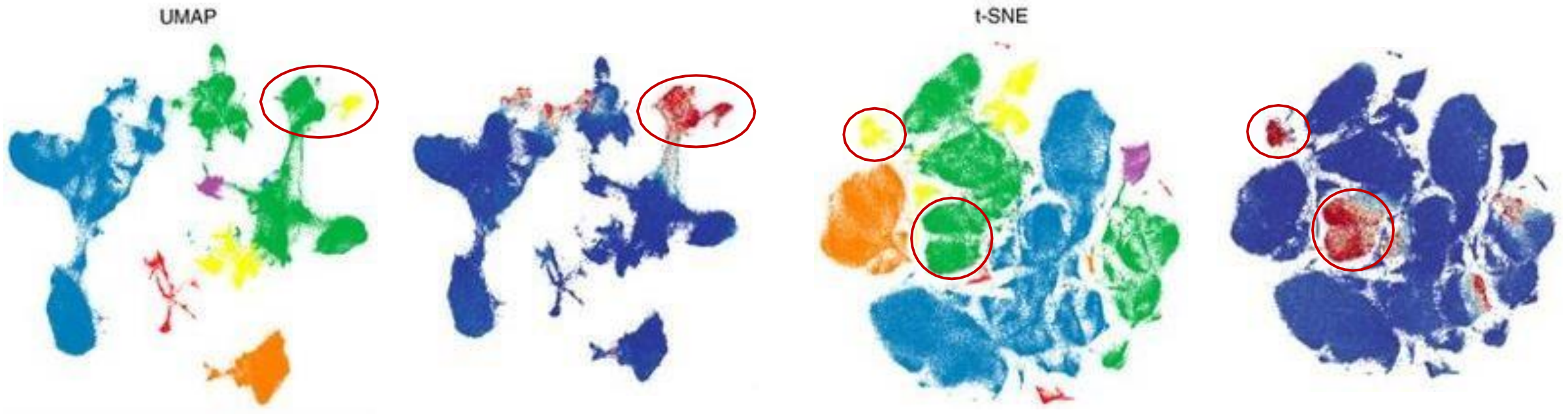


# UMAP vs t-SNE on continuous classes





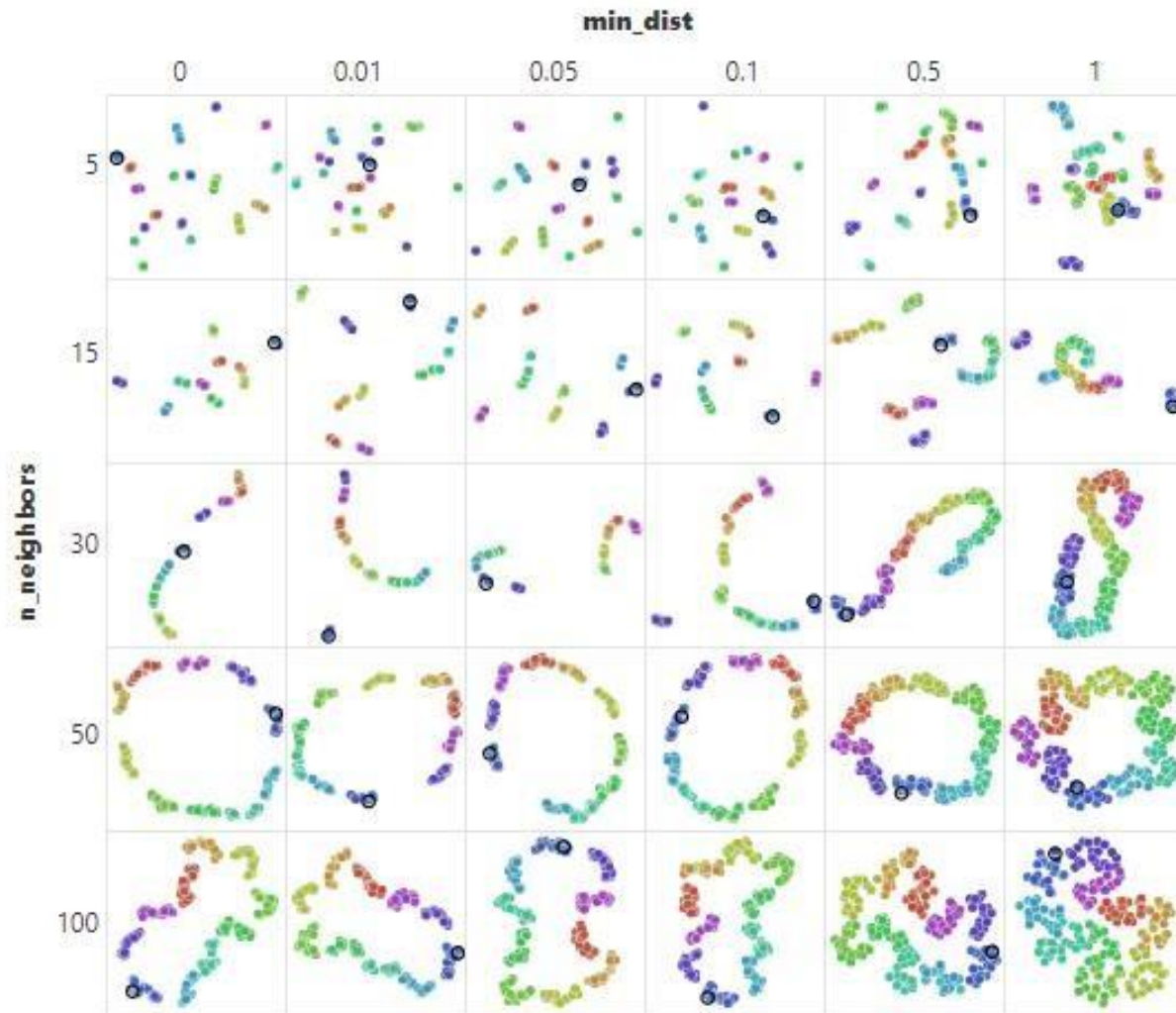
# t-SNE vs UMAP on biological data



Becht, E. et al. Nature Biotechnology 37:38-44 (2019)

- Both are equally good at detecting individual data groups
- But UMAP is better at capturing transitions across data groups

# Customizing UMAP outputs



- Number of neighbors (**n\_neighbors**) is perplexity
- Minimum distant for placing similar data point (**min\_dist**) is for adjusting the scale of visualization



# UMAP in Python

```
class umap.umap_.UMAP(n_neighbors=15, n_components=2, metric='euclidean', metric_kws=None,
output_metric='euclidean', output_metric_kws=None, n_epochs=None, learning_rate=1.0, init='spectral',
min_dist=0.1, spread=1.0, low_memory=True, n_jobs=-1, set_op_mix_ratio=1.0, local_connectivity=1.0,
repulsion_strength=1.0, negative_sample_rate=5, transform_queue_size=4.0, a=None, b=None,
random_state=None, angular_rp_forest=False, target_n_neighbors=-1, target_metric='categorical',
target_metric_kws=None, target_weight=0.5, transform_seed=42, transform_mode='embedding',
force_approximation_algorithm=False, verbose=False, tqdm_kws=None, unique=False, densmap=False,
dens_lambda=2.0, dens_frac=0.3, dens_var_shift=0.1, output_dens=False, disconnection_distance=None,
precomputed_knn=(None, None, None)) [source]
```

- **n\_components** = number of dimensions to embed onto
- **n\_neighbors** = number of nearby data points to use when calculating neighbor graph
  - Recommend trying several values based on dataset size
- **min\_dist** = minimum distance between embedded data points
  - For visualization purpose
- **metric** (see <https://umap-learn.readthedocs.io/en/latest/api.html>)

# UMAP usage

```
!pip install umap-learn from  
umap import UMAP
```

```
## fit on a dataset
```

```
umap = UMAP(n_components = 2, n_neighbors = 25, metric = 'euclidean', min_dist = 0.5,  
random_state = 25).fit(input_data)
```

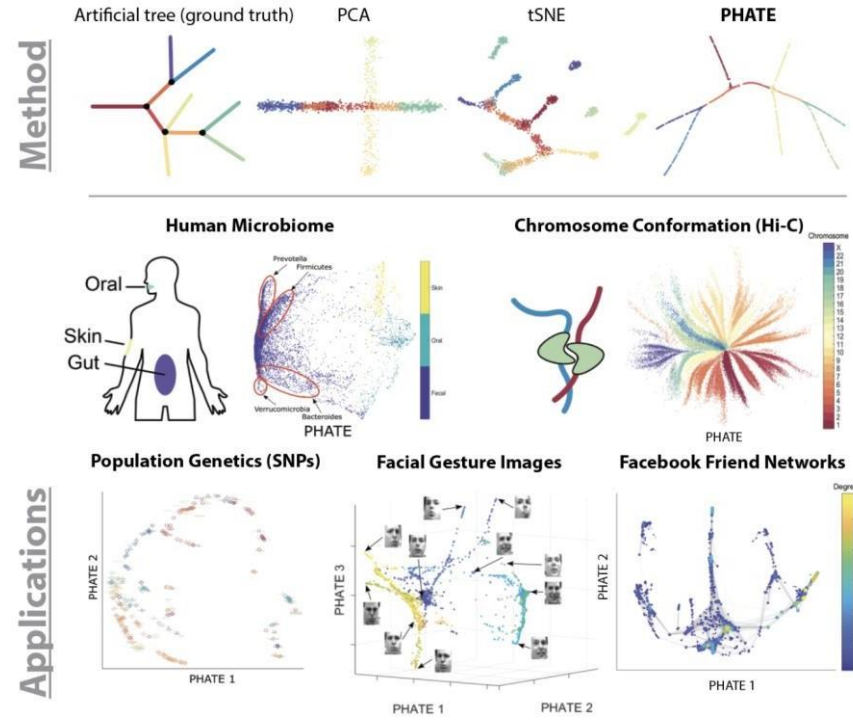
```
## apply to any dataset
```

```
input_embed = umap.transform(input_data)  
output_embed = umap.transform(output_data)
```

# Other improvements

PHATE (<https://www.krishnaswamylab.org/projects/phate>)

Learn local structure/distance (just like UMAP) and global structures (using diffusion maps to traverse the graph). Probably better, but not popular yet.



## DATA

5 tensors found

Word2Vec 10K

Label by

word

Color by

No color map

Edit by

word

Tag selection as

Load

Publish

Download

Label

☒ Sphereize data

Checkpoint: Demo datasets

Metadata: oss\_data/word2vec\_10000\_200d

UMAP

T-SNE

PCA

CUSTOM

Dimension

2D

3D

Neighbors

15

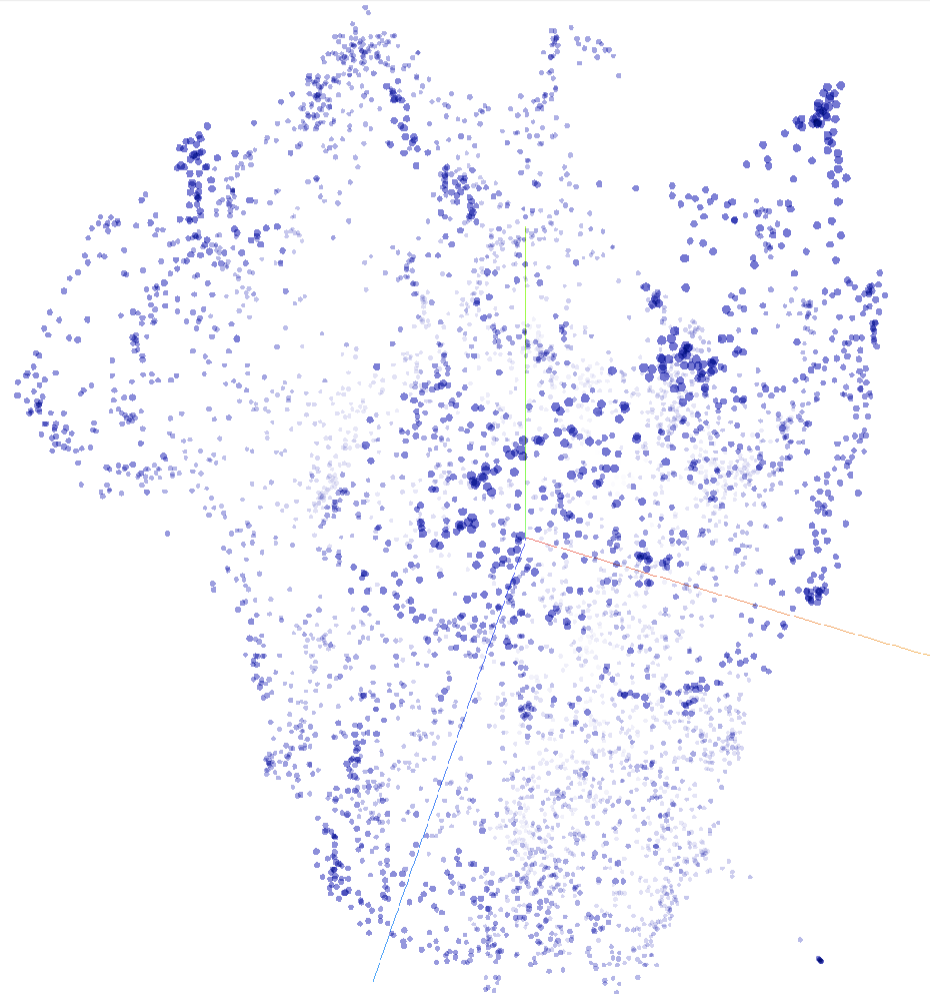
Run

For faster results, the data will be sampled down to 5,000 points.

[Learn more about UMAP.](#)



Points: 10000 | Dimension: 200



<https://projector.tensorflow.org/>

# Demo

- <https://drive.google.com/file/d/1NNrTHXYg88hZwK2wb4-E4EiAON3xOVB3/view?usp=sharing>
- Dataset
  - <https://drive.google.com/drive/folders/1zQZth7T8LJeVbMVv1d6roxG4BD18DpUO?usp=sharing>